i will give you leetcode questions you have to do
1. explain the questions in simple points
2.explain the brute force approach in points with time complexity and space
3.give the code in c++ of that approach
4. explain better approach in points with time complexity and space
5.give the code in c++ of that approach
6. explain optimal approach in points time complexity and space
7.give the code in c++ of that approach

Understood.

For every LeetCode/DSA problem you send, I will follow exactly this structure:

1. Problem Explanation (Simple English)

- What is given?
- What do we need to find?
- Example intuition.
- Why brute force may fail.

2. Brute Force Approach

Idea

- Step-by-step explanation in points.

Time Complexity

- `O(...)`

Space Complexity

- `O(...)`

C++ Code

```cpp
// Brute Force Solution
```

3. Better Approach

Idea

- Optimization over brute force.
- Key observation.
- Step-by-step explanation.

Time Complexity

- `O(...)`

Space Complexity

- `O(...)`

C++ Code

```cpp
// Better Solution
```

4. Optimal Approach

Core Observation

- The trick/important insight.

Idea

- Step-by-step explanation.

Edge Cases

- Mention important corner cases.

Time Complexity

- `O(...)`

Space Complexity

- `O(...)`

C++ Code

```cpp
// Optimal Solution
```

5. Dry Run

- Small example.
- Show variable changes step-by-step.
- Explain how the answer is produced.

6. If the Problem is Hard (1800+/LeetCode Hard)

Additionally:

- Hidden observation/trick.
- What beginners usually miss.
- Alternative ways to think about the problem.

Send the first LeetCode problem.

Group Anagrams

Group Anagrams (LeetCode 49)

1. Problem Explanation (Simple English)

What is given?

- An array of strings `strs`.

Example:

```
["eat","tea","tan","ate","nat","bat"]
```

What do we need to find?

Group together all strings that are anagrams of each other.

Two strings are anagrams if they contain exactly the same characters with the same frequencies.

Example:

```
"eat", "tea", "ate"
```

all belong to the same group.

Expected Output

```
[
  ["eat","tea","ate"],
  ["tan","nat"],
  ["bat"]
]
```

Why brute force may fail?

For every string, checking against every other string becomes expensive when the number of strings is large.

---

2. Brute Force Approach

Idea

- Maintain a visited array.
- Pick a string.
- Compare it with every unvisited string.
- Two strings are anagrams if:
  - Lengths are same.
  - Character frequencies are same.
- Put all matching strings into one group.

Time Complexity

Let:

- `n = number of strings`
- `k = average length of string`

Comparing two strings:

```
O(k)
```

All pair comparisons:

```
O(n² * k)
```

Space Complexity

```
O(n)
```

(for visited array)

C++ Code

```C++
```

```cpp
class Solution {
public:
    bool isAnagram(string &a, string &b) {
        if (a.size() != b.size()) return false;

        vector<int> freq(26, 0);

        for (char c : a) freq[c - 'a']++;
        for (char c : b) freq[c - 'a']--;

        for (int x : freq) {
            if (x != 0) return false;
        }

        return true;
    }

    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        int n = strs.size();

        vector<bool> vis(n, false);
        vector<vector<string>> ans;

        for (int i = 0; i < n; i++) {
            if (vis[i]) continue;

            vector<string> grp;
            grp.push_back(strs[i]);
            vis[i] = true;

            for (int j = i + 1; j < n; j++) {
                if (!vis[j] && isAnagram(strs[i], strs[j])) {
                    grp.push_back(strs[j]);
                    vis[j] = true;
                }
            }

            ans.push_back(grp);
        }

        return ans;
    }
};
```

---

3. Better Approach

Idea

Observation:

- If two strings are anagrams,
- Their sorted forms will be identical.

Example:

```
eat -> aet
tea -> aet
ate -> aet
```

Use:

```cpp
unordered_map<string, vector<string>>
```

where:

```
key = sorted string
value = all strings having that sorted form
```

Time Complexity

Sorting one string:

```
O(k log k)
```

For n strings:

```
O(n * k log k)
```

Space Complexity

```
O(n * k)
```

C++ Code

```cpp
```

```cpp
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {

        unordered_map<string, vector<string>> mp;

        for (string s : strs) {
            string temp = s;
            sort(temp.begin(), temp.end());

            mp[temp].push_back(s);
        }

        vector<vector<string>> ans;

        for (auto &it : mp) {
            ans.push_back(it.second);
        }

        return ans;
    }
};
```

---

4. Optimal Approach

Core Observation

Sorting is unnecessary.

Since strings contain lowercase English letters only:

```
a-z
```

Store character frequencies directly.

Example:

```
eat
```

Frequency:

```
a:1 e:1 t:1
```

Create a unique key from the frequency array.

Example:

```
#1#0#0#0#1...#1...
```

All anagrams generate exactly the same key.

Idea

- For every string:
  - Build frequency array of size 26.
  - Convert frequency array into a unique string key.
  - Store in hashmap.
- Strings with same frequency key belong to same group.

No sorting required.

Time Complexity

Building frequency:

```
O(k)
```

For all strings:

```
O(n * k)
```

Space Complexity

```
O(n * k)
```

C++ Code

```
</> C++
```

```cpp
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {

        unordered_map<string, vector<string>> mp;

        for (string &s : strs) {

            vector<int> freq(26, 0);

            for (char c : s) {
                freq[c - 'a']++;
            }

            string key;

            for (int x : freq) {
                key += "#" + to_string(x);
            }

            mp[key].push_back(s);
        }

        vector<vector<string>> ans;

        for (auto &it : mp) {
            ans.push_back(it.second);
        }

        return ans;
    }
};
```

---

5. Dry Run

Input:

```
["eat","tea","tan","ate","nat","bat"]
```

eat

Frequency key:

```
#1#0#0...#1...#1...
```

Map:

```
key1 -> [eat]
```

---

tea

Same frequency key.

```
key1 -> [eat, tea]
```

---

ate

Same key.

```
key1 -> [eat, tea, ate]
```

---

tan

New key.

```
key2 -> [tan]
```

---

nat

Same key as tan.

```
key2 -> [tan, nat]
```

---

bat

New key.

```
key3 -> [bat]
```

Final Answer

```
[
 [eat, tea, ate],
 [tan, nat],
 [bat]
]
```

What Beginners Usually Miss

1. Sorting is not the only way.
2. Frequency array uniquely identifies an anagram group.
3. Frequency-based solution improves:

```
O(n * k log k)
```

to

```
O(n * k)
```

which is the optimal solution for this problem.

Sources

Sort an Array

Sort an Array (LeetCode 912)

1. Problem Explanation (Simple English)

What is given?

- An integer array `nums`.

Example:

```
nums = [5,2,3,1]
```

What do we need to do?

- Sort the array in ascending order.

Output:

```
[1,2,3,5]
```

Constraints Intuition

- `n` can be up to `50000`.
- Bubble Sort / Selection Sort (`O(n²)`) will be too slow.
- We need an `O(n log n)` sorting algorithm.

---

2. Brute Force Approach

Idea (Bubble Sort)

- Compare adjacent elements.
- Swap if they are in the wrong order.
- Repeat until the array becomes sorted.

Example:

```
5 2 3 1
↓
2 5 3 1
↓
2 3 5 1
↓
2 3 1 5
```

Time Complexity

```
O(n²)
```

Space Complexity

```
O(1)
```

C++ Code

C++

```cpp
class Solution {
public:
    vector<int> sortArray(vector<int>& nums) {
        int n = nums.size();

        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (nums[j] > nums[j + 1]) {
                    swap(nums[j], nums[j + 1]);
                }
            }
        }

        return nums;
    }
};
```

---

3. Better Approach

Idea (Merge Sort)

Observation

- Divide the array into two halves.
- Sort both halves recursively.
- Merge the sorted halves.

Example:

```
[5,2,3,1]

      [5,2,3,1]
      /       \
   [5,2]     [3,1]
   /  \       /  \
 [5] [2]   [3] [1]
```

Then merge back in sorted order.

Time Complexity

```
O(n log n)
```

Space Complexity

```
O(n)
```

(auxiliary merge array)

C++ Code

```C++
```

```cpp
class Solution {
public:
    void merge(vector<int>& nums, int l, int mid, int r) {
        vector<int> temp;

        int i = l;
        int j = mid + 1;

        while (i <= mid && j <= r) {
            if (nums[i] <= nums[j]) {
                temp.push_back(nums[i++]);
            } else {
                temp.push_back(nums[j++]);
            }
        }

        while (i <= mid) {
            temp.push_back(nums[i++]);
        }

        while (j <= r) {
            temp.push_back(nums[j++]);
        }

        for (int k = l; k <= r; k++) {
            nums[k] = temp[k - l];
        }
    }

    void mergeSort(vector<int>& nums, int l, int r) {
        if (l >= r) return;

        int mid = l + (r - l) / 2;

        mergeSort(nums, l, mid);
        mergeSort(nums, mid + 1, r);

        merge(nums, l, mid, r);
    }

    vector<int> sortArray(vector<int>& nums) {
        mergeSort(nums, 0, nums.size() - 1);
        return nums;
    }
};
```

---

4. Optimal Approach

Core Observation

LeetCode expects an efficient sorting algorithm.

The best practical approach is **Quick Sort** with randomized pivot.

Why?

- Average case:

```
O(n log n)
```

- Space:

```
O(log n)
```

(recursion stack)

- Faster in practice than Merge Sort because no extra merge array is required.

Idea

1. Choose a pivot.
2. Put smaller elements on the left.
3. Put larger elements on the right.
4. Recursively sort both parts.

Example:

```
[5,2,3,1]

Pivot = 3

[2,1] 3 [5]

Sort left:
[1,2]

Final:
[1,2,3,5]
```

Time Complexity

Average:

```
O(n log n)
```

Worst:

```
O(n²)
```

(Random pivot makes worst case very unlikely.)

Space Complexity

```
O(log n)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    int partition(vector<int>& nums, int low, int high) {
        int pivot = nums[high];

        int i = low - 1;

        for (int j = low; j < high; j++) {
            if (nums[j] <= pivot) {
                i++;
                swap(nums[i], nums[j]);
            }
        }

        swap(nums[i + 1], nums[high]);

        return i + 1;
    }

    void quickSort(vector<int>& nums, int low, int high) {
        if (low >= high) return;

        int randomIndex = low + rand() % (high - low + 1);
        swap(nums[randomIndex], nums[high]);

        int pi = partition(nums, low, high);

        quickSort(nums, low, pi - 1);
        quickSort(nums, pi + 1, high);
    }

    vector<int> sortArray(vector<int>& nums) {
        quickSort(nums, 0, nums.size() - 1);
        return nums;
    }
};
```

---

5. Dry Run

Input:

```
[5,2,3,1]
```

First Call

Pivot = 3

After partition:

```
[2,1,3,5]
```

Pivot index:

```
2
```

Left Part

```
[2,1]
```

Pivot = 1

After partition:

```
[1,2]
```

Array:

```
[1,2,3,5]
```

Right Part

```
[5]
```

Already sorted.

Final Answer

```
[1,2,3,5]
```

What Beginners Usually Miss

Merge Sort

- Must create a temporary array during merge.
- Time = `O(n log n)` guaranteed.

Quick Sort

- In-place sorting.
- Average `O(n log n)`.
- Use random pivot to avoid TLE on already sorted arrays.

Interview/LeetCode Preference

- Merge Sort → guaranteed complexity.
- Randomized Quick Sort → best practical performance.
- Both are accepted and considered optimal solutions.

Sources

Sort Colors

Sort Colors (LeetCode 75)

1. Problem Explanation (Simple English)

What is given?

An array `nums` containing only:

```
0, 1, 2
```

where:

- `0` = Red
- `1` = White
- `2` = Blue

Example:

```
nums = [2,0,2,1,1,0]
```

What do we need to do?

Sort the array so that:

```
0s come first,
then 1s,
then 2s.
```

Output:

```
[0,0,1,1,2,2]
```

Constraint Intuition

The follow-up asks for:

- One pass
- Constant extra space

So using a normal sorting algorithm is not the most optimal solution.

---

2. Brute Force Approach

Idea

Use any standard sorting algorithm (Bubble Sort).

- Compare adjacent elements.
- Swap if they are in the wrong order.
- Repeat until sorted.

Time Complexity

```
O(n²)
```

Space Complexity

```
O(1)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    void sortColors(vector<int>& nums) {
        int n = nums.size();

        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (nums[j] > nums[j + 1]) {
                    swap(nums[j], nums[j + 1]);
                }
            }
        }
    }
};
```

---

3. Better Approach

Idea (Counting Sort)

Observation:

Only three possible values exist:

```
0, 1, 2
```

Count how many times each occurs.

Example:

```
[2,0,2,1,1,0]

count0 = 2
count1 = 2
count2 = 2
```

Then overwrite the array.

```
0 0 1 1 2 2
```

Time Complexity

```
O(n)
```

Space Complexity

```
O(1)
```

(Only 3 counters)

C++ Code

```C++
class Solution {
public:
    void sortColors(vector<int>& nums) {

        int cnt0 = 0;
        int cnt1 = 0;
        int cnt2 = 0;

        for (int x : nums) {
            if (x == 0) cnt0++;
            else if (x == 1) cnt1++;
            else cnt2++;
        }

        int idx = 0;

        while (cnt0--) nums[idx++] = 0;
        while (cnt1--) nums[idx++] = 1;
        while (cnt2--) nums[idx++] = 2;
    }
};
```

---

4. Optimal Approach

Core Observation

Since there are only three values:

```
0, 1, 2
```

we can place them in their correct regions while traversing only once.

This is called the **Dutch National Flag Algorithm**.

Maintained Regions

```
[0 ... low-1]      => all 0s

[low ... mid-1]    => all 1s

[mid ... high]     => unknown

[high+1 ... n-1]   => all 2s
```

Idea

Maintain:

```cpp
low = 0;
mid = 0;
high = n-1;
```

Case 1: nums[mid] == 0

Put it in the 0-region.

```cpp
swap(nums[low], nums[mid]);
low++;
mid++;
```

Case 2: nums[mid] == 1

Already in correct region.

```cpp
mid++;
```

Case 3: nums[mid] == 2

Put it in the 2-region.

```cpp
```

```
swap(nums[mid], nums[high]);
high--;
```

Do NOT increment `mid` because the swapped element is still unchecked.

Time Complexity

```
O(n)
```

Single traversal.

Space Complexity

```
O(1)
```

C++ Code

```
C++
```

```cpp
class Solution {
public:
    void sortColors(vector<int>& nums) {

        int low = 0;
        int mid = 0;
        int high = nums.size() - 1;

        while (mid <= high) {

            if (nums[mid] == 0) {
                swap(nums[low], nums[mid]);
                low++;
                mid++;
            }
            else if (nums[mid] == 1) {
                mid++;
            }
            else {
                swap(nums[mid], nums[high]);
                high--;
            }
        }
    }
};
```

---

5. Dry Run

Input:

```
[2,0,2,1,1,0]
```

Initially:

```
low = 0
mid = 0
high = 5
```

Step 1

```
nums[mid] = 2
```

Swap with high.

```
[0,0,2,1,1,2]

high = 4
mid = 0
```

Step 2

```
nums[mid] = 0
```

Swap low and mid.

```
[0,0,2,1,1,2]

low = 1
mid = 1
```

Step 3

```
nums[mid] = 0
```

```
[0,0,2,1,1,2]

low = 2
mid = 2
```

Step 4

```
nums[mid] = 2
```

Swap with high.

```
[0,0,1,1,2,2]

high = 3
mid = 2
```

Step 5

```
nums[mid] = 1
```

```
mid = 3
```

Step 6

```
nums[mid] = 1
```

```
mid = 4
```

Now:

```
mid > high
```

Stop.

Final:

```
[0,0,1,1,2,2]
```

What Beginners Usually Miss

Important Trick

After handling a `2`:

```cpp
swap(nums[mid], nums[high]);
high--;
```

Do NOT do:

```cpp
mid++;
```

because the new value at `mid` has not been processed yet.

This single observation is the key to the Dutch National Flag Algorithm and makes the solution work in **one pass with O(1) extra space**.

Sources

Top K Frequent Elements

Top K Frequent Elements (LeetCode 347)

1. Problem Explanation (Simple English)

What is given?

An integer array `nums` and an integer `k`.

Example:

```
nums = [1,1,1,2,2,3]
k = 2
```

What do we need to find?

Return the `k` most frequent elements.

For the above example:

```
1 appears 3 times
2 appears 2 times
3 appears 1 time
```

Top 2 frequent elements:

```
[1,2]
```

Constraint Intuition

If there are `n` elements:

- Counting frequencies is easy.
- Sorting all frequencies costs `O(n log n)`.
- Follow-up asks for better than `O(n log n)`.

---

2. Brute Force Approach

Idea

Step 1

Count frequencies using hashmap.

```
1 -> 3
2 -> 2
3 -> 1
```

Step 2

Repeatedly find the maximum frequency element.

Step 3

Add it to answer and remove it.

Step 4

Repeat `k` times.

Time Complexity

Let `m` = distinct elements.

Finding max frequency:

```
O(m)
```

Done `k` times:

```
O(k * m)
```

Worst case:

```
O(n²)
```

Space Complexity

```
O(m)
```

C++ Code

C++

```cpp
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {

        unordered_map<int,int> mp;

        for(int x : nums)
            mp[x]++;

        vector<int> ans;

        while(k--) {

            int mxFreq = 0;
            int element = 0;

            for(auto &it : mp) {
                if(it.second > mxFreq) {
                    mxFreq = it.second;
                    element = it.first;
                }
            }

            ans.push_back(element);
            mp.erase(element);
        }

        return ans;
    }
};
```

3. Better Approach

Idea (Sorting Frequencies)

Step 1

Count frequencies.

```
1 -> 3
2 -> 2
3 -> 1
```

Step 2

Store:

```
(freq, element)

(3,1)
(2,2)
(1,3)
```

Step 3

Sort in descending frequency.

Step 4

Take first `k` elements.

Time Complexity

Frequency count:

```
O(n)
```

Sorting:

```
O(m log m)
```

Overall:

```
O(n + m log m)
```

Worst case:

```
O(n log n)
```

Space Complexity

```
O(m)
```

C++ Code

```C++
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {

        unordered_map<int,int> mp;

        for(int x : nums)
            mp[x]++;

        vector<pair<int,int>> vec;

        for(auto &it : mp) {
            vec.push_back({it.second, it.first});
        }

        sort(vec.begin(), vec.end(), greater<pair<int,int>>());

        vector<int> ans;

        for(int i = 0; i < k; i++) {
            ans.push_back(vec[i].second);
        }

        return ans;
    }
};
```

---

4. Optimal Approach

Core Observation

Frequency of any element can be at most:

```
n
```

where `n = nums.size()`.

Instead of sorting elements, place them according to frequency.

This is called **Bucket Sort**.

Idea

Step 1

Count frequencies.

```
1 -> 3
2 -> 2
3 -> 1
```

Step 2

Create buckets:

```C++
vector<vector<int>> bucket(n + 1);
```

Index = frequency.

Example:

```
bucket[1] = [3]
bucket[2] = [2]
bucket[3] = [1]
```

Step 3

Traverse buckets from back.

```
freq = n -> 1
```

Collect elements until `k` elements are obtained.

Time Complexity

Frequency count:

```
O(n)
```

Bucket traversal:

```
O(n)
```

Overall:

```
O(n)
```

Space Complexity

```
O(n)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {

        unordered_map<int,int> mp;

        for(int x : nums)
            mp[x]++;

        int n = nums.size();

        vector<vector<int>> bucket(n + 1);

        for(auto &it : mp) {
            bucket[it.second].push_back(it.first);
        }

        vector<int> ans;

        for(int freq = n; freq >= 1; freq--) {

            for(int num : bucket[freq]) {

                ans.push_back(num);

                if(ans.size() == k)
                    return ans;
            }
        }

        return ans;
    }
};
```

---

5. Dry Run

Input:

```
nums = [1,1,1,2,2,3]
k = 2
```

Frequency Map

```
1 -> 3
2 -> 2
3 -> 1
```

Buckets

```
bucket[1] = [3]

bucket[2] = [2]

bucket[3] = [1]
```

Traverse Backwards

Start from frequency 6:

```
bucket[6] empty
bucket[5] empty
bucket[4] empty
```

Frequency 3:

```
bucket[3] = [1]

ans = [1]
```

Frequency 2:

```
bucket[2] = [2]

ans = [1,2]
```

Size becomes `k`.

Return:

```
[1,2]
```

What Beginners Usually Miss

Observation 1

We never need to sort the actual numbers.

We only care about:

```
frequency → elements
```

Observation 2

Frequency is bounded by:

```
1 ≤ frequency ≤ n
```

which makes Bucket Sort possible.

Observation 3

Another accepted optimal solution is using a **Min Heap**.

- Time: `O(n log k)`
- Space: `O(k)`

This is often preferred in interviews when `k` is much smaller than `n`.

However, for LeetCode 347, **Bucket Sort** (`O(n)`) is the optimal solution.

Sources

Encode and Decode Strings Encode and Decode Strings

Encode and Decode Strings (LeetCode Premium)

1. Problem Explanation (Simple English)

What is given?

A list of strings:

```
["neet","code","love","you"]
```

What do we need to do?

Implement two functions:

Encode

Convert the list into a single string.

```
["neet","code","love","you"]
↓
encoded_string
```

Decode

Convert the encoded string back into the original list.

```
encoded_string
↓
["neet","code","love","you"]
```

Important

The encoded string should work even if strings contain:

```
":", "#", "/", digits, spaces, etc.
```

Example:

```
["ab#c","xy"]
```

must decode correctly.

---

2. Brute Force Approach

Idea

Use a delimiter.

Example:

```
["neet","code","love"]
```

Encode as:

```
neet|code|love
```

During decoding:

```
</> C++
```

```
split('|')
```

Problem

Fails when strings themselves contain:

```
"|"
```

Example:

```
["ab|cd","xy"]
```

Encoded:

```
ab|cd|xy
```

Now we cannot determine:

```
["ab","cd","xy"]
```

or

```
["ab|cd","xy"]
```

Time Complexity

Encode:

```
O(total_characters)
```

Decode:

```
O(total_characters)
```

Space Complexity

```
O(total_characters)
```

C++ Code

```C++
class Codec {
public:
    string encode(vector<string>& strs) {
        string res;

        for (int i = 0; i < strs.size(); i++) {
            res += strs[i];

            if (i + 1 < strs.size()) {
                res += '|';
            }
        }

        return res;
    }

    vector<string> decode(string s) {
        vector<string> ans;
        string cur;

        for (char ch : s) {
            if (ch == '|') {
                ans.push_back(cur);
                cur.clear();
            } else {
                cur += ch;
            }
        }

        ans.push_back(cur);

        return ans;
    }
};
```

---

3. Better Approach

Idea

Escape special characters.

Example:

Before encoding:

```
ab|cd
```

Convert:

```
ab\|cd
```

Then join using `|`.

During decoding:

- Handle escaped characters separately.

Time Complexity

Encode:

```
O(n)
```

Decode:

```
O(n)
```

where `n = total characters`.

Space Complexity

```
O(n)
```

C++ Code

```C++
```

```cpp
class Codec {
public:
    string encode(vector<string>& strs) {

        string res;

        for (int i = 0; i < strs.size(); i++) {

            for (char ch : strs[i]) {
                if (ch == '|' || ch == '\\')
                    res += '\\';

                res += ch;
            }

            if (i + 1 < strs.size())
                res += '|';
        }

        return res;
    }

    vector<string> decode(string s) {

        vector<string> ans;
        string cur;

        bool escape = false;

        for (char ch : s) {

            if (escape) {
                cur += ch;
                escape = false;
            }
            else if (ch == '\\') {
                escape = true;
            }
            else if (ch == '|') {
                ans.push_back(cur);
                cur.clear();
            }
            else {
                cur += ch;
            }
        }

        ans.push_back(cur);

        return ans;
    }
};
```

4. Optimal Approach

Core Observation

Instead of relying on delimiters, store:

```
length + delimiter + string
```

Example:

```
"neet"
```

becomes:

```
4#neet
```

and

```
"code"
```

becomes:

```
4#code
```

Encoded string:

```
4#neet4#code4#love3#you
```

Why does this always work?

When decoding:

1. Read length before `#`.
2. Extract exactly that many characters.
3. Move to the next string.

Even if the string contains:

```
#
###
abc#xyz
```

it still works.

Idea

Encode

For every string:

```C++
encoded += to_string(length) + "#" + string;
```

Decode

For each block:

1. Find `#`
2. Read length
3. Take next `length` characters
4. Continue

Time Complexity

Encode:

```
O(n)
```

Decode:

```
O(n)
```

where `n = total characters`.

Space Complexity

```
O(n)
```

C++ Code

C++

```cpp
class Codec {
public:

    string encode(vector<string>& strs) {

        string res;

        for (string &s : strs) {
            res += to_string(s.size()) + "#" + s;
        }

        return res;
    }

    vector<string> decode(string s) {

        vector<string> ans;

        int i = 0;

        while (i < s.size()) {

            int j = i;

            while (s[j] != '#')
                j++;

            int len = stoi(s.substr(i, j - i));

            j++;

            ans.push_back(s.substr(j, len));

            i = j + len;
        }

        return ans;
    }
};
```

---

5. Dry Run

Input:

```
["neet","code","love"]
```

Encoding

For `"neet"`

```
4#neet
```

For `"code"`

```
4#code
```

For `"love"`

```
4#love
```

Final:

```
4#neet4#code4#love
```

Decoding

Start:

```
4#neet4#code4#love
^
```

Read length:

```
4
```

Take next 4 chars:

```
neet
```

Move ahead.

Read:

```
4#code
```

Length:

```
4
```

Take:

```
code
```

Read:

```
4#love
```

Length:

```
4
```

Take:

```
love
```

Result:

```
["neet","code","love"]
```

What Beginners Usually Miss

Mistake 1

Using a delimiter only:

```
a|b|c
```

This breaks if strings contain `|`.

Mistake 2

Assuming `#` cannot appear in strings.

The length-based approach does **not care** what characters are inside the string.

Key Trick

Store:

```
length#string
```

instead of:

```
delimiter-separated strings
```

This guarantees **unique and lossless decoding**, making it the standard optimal solution used in interviews.

Range Sum Query 2D Immutable

Range Sum Query 2D - Immutable (LeetCode 304)

1. Problem Explanation (Simple English)

What is given?

A 2D matrix:

```
[
 [3,0,1,4,2],
 [5,6,3,2,1],
 [1,2,0,1,5],
 [4,1,0,1,7],
 [1,0,3,0,5]
]
```

You need to answer many queries:

```C++
sumRegion(row1, col1, row2, col2)
```

which asks for the sum of all elements inside that rectangle.

Example:

```C++
sumRegion(2,1,4,3)
```

Rectangle:

```
2 0 1
1 0 1
0 3 0
```

Answer:

```
8
```

What is required?

Implement:

```C++
NumMatrix(matrix)
```

and

```C++
sumRegion(row1,col1,row2,col2)
```

efficiently.

Constraint Intuition

There can be many queries.

If we calculate the rectangle sum from scratch every time, it becomes slow.

We need preprocessing.

---

2. Brute Force Approach

Idea

For every query:

- Traverse all cells inside the rectangle.
- Add them to the answer.

Example:

```C++
for(int i=row1;i<=row2;i++)
    for(int j=col1;j<=col2;j++)
        sum += matrix[i][j];
```

Time Complexity

For one query:

```
O((row2-row1+1)*(col2-col1+1))
```

Worst case:

```
O(m*n)
```

For Q queries:

```
O(Q*m*n)
```

Space Complexity

```
O(1)
```

C++ Code

C++

```cpp
class NumMatrix {
public:
    vector<vector<int>> mat;

    NumMatrix(vector<vector<int>>& matrix) {
        mat = matrix;
    }

    int sumRegion(int row1, int col1, int row2, int col2) {

        int sum = 0;

        for(int i = row1; i <= row2; i++) {
            for(int j = col1; j <= col2; j++) {
                sum += mat[i][j];
            }
        }

        return sum;
    }
};
```

---

3. Better Approach

Idea (Row Prefix Sum)

Precompute prefix sum for every row.

For each row:

```
[3,0,1,4,2]

prefix:

[3,3,4,8,10]
```

Now rectangle query:

For each row:

C++

```
rowSum =
prefix[row][col2]
-
prefix[row][col1-1]
```

Add all rows.

Time Complexity

Preprocessing:

```
O(m*n)
```

Query:

```
O(number of rows)
```

```
O(m)
```

Space Complexity

```
O(m*n)
```

C++ Code

```
C++
```

```
class NumMatrix {
public:

    vector<vector<int>> pref;

    NumMatrix(vector<vector<int>>& matrix) {

        int m = matrix.size();
        int n = matrix[0].size();

        pref = vector<vector<int>>(m, vector<int>(n));

        for(int i = 0; i < m; i++) {

            pref[i][0] = matrix[i][0];

            for(int j = 1; j < n; j++) {
                pref[i][j] = pref[i][j - 1] + matrix[i][j];
            }
        }
    }

    int sumRegion(int row1, int col1, int row2, int col2) {

        int ans = 0;

        for(int row = row1; row <= row2; row++) {

            ans += pref[row][col2];

            if(col1 > 0)
                ans -= pref[row][col1 - 1];
        }

        return ans;
    }
};
```

---

4. Optimal Approach

Core Observation

This is a classic **2D Prefix Sum** problem.

Instead of storing prefix for each row separately, store prefix for the entire matrix.

Let:

```
pref[i][j]
=
sum of all elements
from (0,0) to (i-1,j-1)
```

We use a `(m+1) x (n+1)` matrix.

Formula

For building prefix:

```
pref[i][j]
=
matrix[i-1][j-1]
+ pref[i-1][j]
+ pref[i][j-1]
- pref[i-1][j-1]
```

Query Formula

Rectangle:

```
(row1,col1) -> (row2,col2)
```

Answer:

```
pref[row2+1][col2+1]
- pref[row1][col2+1]
- pref[row2+1][col1]
+ pref[row1][col1]
```

This is Inclusion-Exclusion.

Time Complexity

Constructor

```
O(m*n)
```

Each Query

```
O(1)
```

Space Complexity

```
O(m*n)
```

C++ Code

C++

```cpp
class NumMatrix {
public:

    vector<vector<int>> pref;

    NumMatrix(vector<vector<int>>& matrix) {

        int m = matrix.size();
        int n = matrix[0].size();

        pref = vector<vector<int>>(m + 1, vector<int>(n + 1, 0));

        for(int i = 1; i <= m; i++) {
            for(int j = 1; j <= n; j++) {

                pref[i][j] =
                    matrix[i - 1][j - 1]
                    + pref[i - 1][j]
                    + pref[i][j - 1]
                    - pref[i - 1][j - 1];
            }
        }
    }

    int sumRegion(int row1, int col1, int row2, int col2) {

        return pref[row2 + 1][col2 + 1]
             - pref[row1][col2 + 1]
             - pref[row2 + 1][col1]
             + pref[row1][col1];
    }
};
```

5. Dry Run

Matrix:

```
1 2
3 4
```

Build Prefix

Using extra row and column:

```
0 0 0
0 1 3
0 4 10
```

Query:

```C++
sumRegion(0,0,1,1)
```

Entire matrix.

Formula:

```
pref[2][2]
- pref[0][2]
- pref[2][0]
+ pref[0][0]
```

```
10 - 0 - 0 + 0
```

Answer:

```
10
```

Correct.

What Beginners Usually Miss

Observation 1

This is exactly the 2D version of:

```
Range Sum Query - Immutable
```

where we use 1D prefix sums.

Observation 2

Using an extra row and column:

```cpp
m+1
n+1
```

eliminates boundary checks.

Observation 3

The most important formula is:

```
Total
- Upper Rectangle
- Left Rectangle
+ Overlap Rectangle
```

This Inclusion-Exclusion trick makes every query run in:

```
O(1)
```

which is the optimal solution.

Sources

Product of Array Except Self

Product of Array Except Self (LeetCode 238)

1. Problem Explanation (Simple English)

What is given?

An integer array `nums`.

Example:

```
nums = [1,2,3,4]
```

What do we need to find?

For every index `i`, return:

```
product of all elements except nums[i]
```

Example:

```
Index 0:
2*3*4 = 24

Index 1:
1*3*4 = 12

Index 2:
1*2*4 = 8

Index 3:
1*2*3 = 6
```

Output:

```
[24,12,8,6]
```

Constraints

- Do NOT use division.
- Solve in `O(n)` time.
- Follow-up: `O(1)` extra space.

---

2. Brute Force Approach

Idea

For every index:

- Multiply all other elements.
- Skip current index.

Steps

For each `i`:

```C++
product = 1

for every j:
    if(i != j)
        product *= nums[j]
```

Store result.

Time Complexity

Outer loop:

```
O(n)
```

Inner loop:

```
O(n)
```

Total:

```
O(n²)
```

Space Complexity

```
O(1)
```

(ignoring output array)

C++ Code

```C++
```

```cpp
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {

        int n = nums.size();

        vector<int> ans(n);

        for(int i = 0; i < n; i++) {

            long long prod = 1;

            for(int j = 0; j < n; j++) {

                if(i == j) continue;

                prod *= nums[j];
            }

            ans[i] = prod;
        }

        return ans;
    }
};
```

---

3. Better Approach

Idea (Prefix + Suffix Arrays)

Observation

For index `i`:

```
Answer =
(Product of all elements left of i)
×
(Product of all elements right of i)
```

Example:

```
nums = [1,2,3,4]

For index 2:

Left Product = 1*2 = 2
Right Product = 4

Answer = 2*4 = 8
```

Build Prefix

```
prefix[i]
=
product from 0 to i
```

Example:

```
nums = [1,2,3,4]

prefix =
[1,2,6,24]
```

Build Suffix

```
suffix[i]
=
product from i to n-1
```

```
suffix =
[24,24,12,4]
```

Calculate Answer

```
ans[i]
=
prefix[i-1] * suffix[i+1]
```

Time Complexity

```
O(n)
```

Space Complexity

```
O(n)
```

(prefix + suffix)

C++ Code

```C++
```

```cpp
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {

        int n = nums.size();

        vector<int> prefix(n);
        vector<int> suffix(n);
        vector<int> ans(n);

        prefix[0] = nums[0];

        for(int i = 1; i < n; i++) {
            prefix[i] = prefix[i - 1] * nums[i];
        }

        suffix[n - 1] = nums[n - 1];

        for(int i = n - 2; i >= 0; i--) {
            suffix[i] = suffix[i + 1] * nums[i];
        }

        for(int i = 0; i < n; i++) {

            int left = (i == 0) ? 1 : prefix[i - 1];
            int right = (i == n - 1) ? 1 : suffix[i + 1];

            ans[i] = left * right;
        }

        return ans;
    }
};
```

---

4. Optimal Approach

Core Observation

We don't actually need both prefix and suffix arrays.

Use:

- Answer array to store prefix products.
- One variable to maintain suffix product.

This satisfies:

```
O(n) Time
O(1) Extra Space
```

Idea

First Pass

Store left products directly in answer.

```
nums = [1,2,3,4]

ans[0] = 1

ans =
[1,1,2,6]
```

Meaning:

```
ans[i]
=
product of all elements before i
```

Second Pass

Traverse from right.

Maintain:

```C++
suffix = 1
```

At each index:

```C++
ans[i] *= suffix
suffix *= nums[i]
```

Time Complexity

```
O(n)
```

Space Complexity

```
O(1)
```

(Answer array is not counted as extra space)

C++ Code

```C++
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {

        int n = nums.size();

        vector<int> ans(n);

        ans[0] = 1;

        for(int i = 1; i < n; i++) {
            ans[i] = ans[i - 1] * nums[i - 1];
        }

        int suffix = 1;

        for(int i = n - 1; i >= 0; i--) {

            ans[i] *= suffix;

            suffix *= nums[i];
        }

        return ans;
    }
};
```

---

5. Dry Run

Input:

```
nums = [1,2,3,4]
```

First Pass (Prefix)

```
ans[0] = 1

ans[1] = 1*1 = 1

ans[2] = 1*2 = 2

ans[3] = 2*3 = 6
```

Current:

```
ans = [1,1,2,6]
```

Second Pass (Suffix)

Start:

```
suffix = 1
```

---

i = 3

```
ans[3] = 6*1 = 6

suffix = 1*4 = 4
```

---

i = 2

```
ans[2] = 2*4 = 8

suffix = 4*3 = 12
```

---

i = 1

```
ans[1] = 1*12 = 12

suffix = 12*2 = 24
```

---

i = 0

```
ans[0] = 1*24 = 24

suffix = 24*1 = 24
```

Final:

```
[24,12,8,6]
```

What Beginners Usually Miss

Observation 1

For every index:

```
Answer =
Left Product × Right Product
```

Observation 2

You do NOT need division.

This is important because zeros can exist.

Example:

```
[1,2,0,4]
```

Division-based solutions break.

Observation 3 (Key Trick)

Store prefix products in the answer array itself:

```cpp
ans[i] = product before i
```

Then multiply by suffix product while traversing backwards.

This removes the need for a separate suffix array and achieves the optimal:

```
Time  : O(n)
Space : O(1)
```

Sources

Valid Sudoku

Valid Sudoku (LeetCode 36)

1. Problem Explanation (Simple English)

What is given?

A `9 x 9` Sudoku board.

Each cell contains:

```
'1' to '9'
or
'.' (empty cell)
```

Example:

```
[
["5","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
...
]
```

What do we need to check?

The board is valid if:

Rule 1: Row

A digit must not appear twice in the same row.

Example:

```
5 3 5 . . . . . .
```

Invalid because `5` appears twice.

Rule 2: Column

A digit must not appear twice in the same column.

Rule 3: 3×3 Box

A digit must not appear twice inside the same 3×3 sub-box.

Example:

```
5 . .
. 5 .
. . .
```

Invalid.

Important

We only need to check whether the current board is valid.

We do **NOT** need to solve the Sudoku.

---

2. Brute Force Approach

Idea

For every filled cell:

1. Check entire row.

2. Check entire column.
3. Check entire 3×3 box.
4. If the same digit is found again → Invalid.

Time Complexity

For each cell:

```
Check row    -> O(9)
Check column -> O(9)
Check box    -> O(9)
```

Total:

```
81 × 27
```

```
O(81²)
```

For a fixed 9×9 board this is constant, but logically it's the brute-force approach.

Space Complexity

```
O(1)
```

C++ Code

‹/› C++

```cpp
class Solution {
public:
    bool isValidSudoku(vector<vector<char>>& board) {

        for(int r = 0; r < 9; r++) {
            for(int c = 0; c < 9; c++) {

                if(board[r][c] == '.') continue;

                char val = board[r][c];

                // Row
                for(int j = 0; j < 9; j++) {
                    if(j != c && board[r][j] == val)
                        return false;
                }

                // Column
                for(int i = 0; i < 9; i++) {
                    if(i != r && board[i][c] == val)
                        return false;
                }

                // Box
                int sr = (r / 3) * 3;
                int sc = (c / 3) * 3;

                for(int i = sr; i < sr + 3; i++) {
                    for(int j = sc; j < sc + 3; j++) {

                        if((i != r || j != c) &&
                           board[i][j] == val)
                            return false;
                    }
                }
            }
        }

        return true;
    }
};
```

---

3. Better Approach

Idea

Use three hash sets:

Row Set

Store:

```
(row, digit)
```

Example:

```
(0,5)
```

means digit `5` already appeared in row `0`.

Column Set

Store:

```
(col, digit)
```

Box Set

Store:

```
(box, digit)
```

where:

```C++
box = (row/3)*3 + col/3
```

While traversing:

- If digit already exists in any set → Invalid.
- Otherwise insert it.

Time Complexity

Traverse board once:

```
O(81)
```

Space Complexity

```
O(81)
```

C++ Code

C++

```cpp
class Solution {
public:
    bool isValidSudoku(vector<vector<char>>& board) {

        unordered_set<string> seen;

        for(int r = 0; r < 9; r++) {
            for(int c = 0; c < 9; c++) {

                if(board[r][c] == '.')
                    continue;

                string row =
                    "r" + to_string(r) + board[r][c];

                string col =
                    "c" + to_string(c) + board[r][c];

                string box =
                    "b" + to_string((r / 3) * 3 + c / 3) + board[r][c];

                if(seen.count(row) ||
                   seen.count(col) ||
                   seen.count(box))
                    return false;

                seen.insert(row);
                seen.insert(col);
                seen.insert(box);
            }
        }

        return true;
    }
};
```

4. Optimal Approach

Core Observation

There are only:

```
9 rows
9 columns
9 boxes
```

and digits:

```
1 to 9
```

We can directly use boolean arrays instead of hash sets.

This is faster and cleaner.

Idea

Maintain:

```C++
rows[9][9]
cols[9][9]
boxes[9][9]
```

Meaning:

```
rows[r][d]
=
digit d already appeared in row r
```

Similarly for columns and boxes.

Box Number

```C++
box = (r / 3) * 3 + (c / 3)
```

Box indices:

```
0 1 2
3 4 5
6 7 8
```

Process Each Cell

If current digit already exists in:

```cpp
rows[r][num]
or
cols[c][num]
or
boxes[box][num]
```

return false.

Otherwise mark all three.

Time Complexity

```
O(81)
```

Single traversal.

Space Complexity

```
O(1)
```

(9×9 arrays are constant-sized.)

C++ Code

```cpp
```

```cpp
class Solution {
public:
    bool isValidSudoku(vector<vector<char>>& board) {

        bool rows[9][9] = {};
        bool cols[9][9] = {};
        bool boxes[9][9] = {};

        for(int r = 0; r < 9; r++) {
            for(int c = 0; c < 9; c++) {

                if(board[r][c] == '.')
                    continue;

                int num = board[r][c] - '1';

                int box = (r / 3) * 3 + (c / 3);

                if(rows[r][num] ||
                   cols[c][num] ||
                   boxes[box][num])
                    return false;

                rows[r][num] = true;
                cols[c][num] = true;
                boxes[box][num] = true;
            }
        }

        return true;
    }
};
```

---

5. Dry Run

Suppose:

```
[
["5","3","."],
["6",".","."],
[".","9","8"]
]
```

Cell (0,0)

```
digit = 5
num = 4
box = 0
```

Check:

```
rows[0][4]  = false
cols[0][4]  = false
boxes[0][4] = false
```

Mark all as true.

Cell (0,1)

```
digit = 3
num = 2
box = 0
```

Not seen before.

Mark.

If Another 5 Appears in Row 0

Example:

```
5 3 . . . 5
```

Then:

```
rows[0][4] == true
```

Immediately return:

```
false
```

What Beginners Usually Miss

Trick 1

The box number formula:

```cpp
box = (r / 3) * 3 + (c / 3);
```

This is the most important observation.

Example:

```
(4,7)

box = (4/3)*3 + (7/3)
    = 1*3 + 2
    = 5
```

Trick 2

We don't need to compare cells with each other.

Instead, record what has already appeared.

Trick 3

The optimal solution is not about reducing time below `O(81)`.

Since the board size is fixed, the goal is to use the cleanest and fastest bookkeeping method:

```
Rows + Columns + Boxes
```

using boolean arrays.

Sources

Longest Consecutive Sequence

Longest Consecutive Sequence (LeetCode 128)

1. Problem Explanation (Simple English)

What is given?

An integer array `nums`.

Example:

```
nums = [100,4,200,1,3,2]
```

What do we need to find?

Find the length of the longest sequence of consecutive integers.

Example:

```
1,2,3,4
```

is consecutive.

So:

```
[100,4,200,1,3,2]
```

contains:

```
1,2,3,4
```

Answer:

```
4
```

Constraints Intuition

A sorting solution gives:

```
O(n log n)
```

But the problem asks for:

```
O(n)
```

So we need a HashSet-based solution.

---

2. Brute Force Approach

Idea

For every number:

- Try to find:
  - `num + 1`
  - `num + 2`
  - `num + 3`
  - ...
- Continue until the sequence breaks.

To check whether a number exists, scan the entire array.

Time Complexity

For each element:

```
O(n)
```

Searching next elements:

```
O(n)
```

Overall:

```
O(n²)
```

Space Complexity

```
O(1)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    bool exists(vector<int>& nums, int target) {
        for (int x : nums) {
            if (x == target) return true;
        }
        return false;
    }

    int longestConsecutive(vector<int>& nums) {

        int ans = 0;

        for (int num : nums) {

            int len = 1;
            int cur = num;

            while (exists(nums, cur + 1)) {
                cur++;
                len++;
            }

            ans = max(ans, len);
        }

        return ans;
    }
};
```

---

3. Better Approach

Idea (Sorting)

Observation

After sorting, consecutive numbers become adjacent.

Example:

```
[100,4,200,1,3,2]
```

After sorting:

```
[1,2,3,4,100,200]
```

Now count consecutive streaks.

Handle Duplicates

Example:

```
[1,2,2,3]
```

The second `2` should be skipped.

Time Complexity

Sorting:

```
O(n log n)
```

Traversal:

```
O(n)
```

Total:

```
O(n log n)
```

Space Complexity

```
O(1)
```

(ignoring sorting stack)

C++ Code

```C++
```

```cpp
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {

        if(nums.empty()) return 0;

        sort(nums.begin(), nums.end());

        int ans = 1;
        int cur = 1;

        for(int i = 1; i < nums.size(); i++) {

            if(nums[i] == nums[i - 1]) {
                continue;
            }

            if(nums[i] == nums[i - 1] + 1) {
                cur++;
            }
            else {
                ans = max(ans, cur);
                cur = 1;
            }
        }

        ans = max(ans, cur);

        return ans;
    }
};
```

---

4. Optimal Approach

Core Observation

A number should start counting a sequence **only if it is the beginning of that sequence**.

For example:

```
1,2,3,4
```

Start only from:

```
1
```

Do not start from:

```
2,3,4
```

because they are already inside a sequence.

Key Trick

Put all numbers into a HashSet.

For each number:

Check:

```C++
num - 1
```

If it exists:

```
num is NOT the start
```

Skip it.

If it does not exist:

```
num is the start of a sequence
```

Now keep extending:

```C++
num+1
num+2
num+3
...
```

Why is it O(n)?

Each number is visited at most once while extending sequences.

So total work remains linear.

Time Complexity

HashSet insertion:

```
O(n)
```

Traversal:

```
O(n)
```

Overall:

```
O(n)
```

Space Complexity

```
O(n)
```

(HashSet)

C++ Code

```C++
```

```cpp
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {

        unordered_set<int> st(nums.begin(), nums.end());

        int ans = 0;

        for(int num : st) {

            if(st.count(num - 1))
                continue;

            int cur = num;
            int len = 1;

            while(st.count(cur + 1)) {
                cur++;
                len++;
            }

            ans = max(ans, len);
        }

        return ans;
    }
};
```

---

5. Dry Run

Input:

```
nums = [100,4,200,1,3,2]
```

HashSet:

```
{100,4,200,1,3,2}
```

---

num = 100

Check:

```
99 exists?
No
```

Start sequence:

```
100
```

Length:

```
1
```

Answer:

```
1
```

---

num = 4

Check:

```
3 exists?
Yes
```

Skip.

---

num = 200

Check:

```
199 exists?
No
```

Sequence:

```
200
```

Length:

```
1
```

---

num = 1

Check:

```
0 exists?
No
```

Start sequence:

```
1 → 2 → 3 → 4
```

Length:

```
4
```

Answer:

```
4
```

---

num = 2

Check:

```
1 exists?
Yes
```

Skip.

---

num = 3

Check:

```
2 exists?
Yes
```

Skip.

Final:

```
4
```

What Beginners Usually Miss

Observation 1

The crucial trick is:

```cpp
if(st.count(num - 1))
    continue;
```

This ensures only sequence starters are processed.

Observation 2

Without this check:

```
1 → 2 → 3 → 4
2 → 3 → 4
3 → 4
4
```

which becomes:

```
O(n²)
```

Observation 3

The problem specifically asks for:

```
O(n)
```

The HashSet + Sequence Starter approach is the standard optimal solution.

Final Complexity

```
Time  : O(n)
Space : O(n)
```

Sources

Best Time to Buy And Sell Stock II

Best Time to Buy and Sell Stock II (LeetCode 122)

1. Problem Explanation (Simple English)

What is given?

An array `prices` where:

```
prices[i]
```

is the stock price on day `i`.

Example:

```
prices = [7,1,5,3,6,4]
```

What do we need to find?

You can:

- Buy and sell multiple times.
- Hold at most one stock at a time.
- You may buy and sell on the same day.

Return the maximum profit.

Example:

```
Buy at 1, Sell at 5  => Profit = 4

Buy at 3, Sell at 6  => Profit = 3

Total Profit = 7
```

Output:

```
7
```

Constraint Intuition

Unlike Stock I:

```
Only one transaction
```

Here:

```
Unlimited transactions allowed
```

This changes the solution completely.

---

2. Brute Force Approach

Idea

At every day:

Option 1

Do nothing.

Option 2

Buy stock (if not holding).

Option 3

Sell stock (if holding).

Try all possibilities using recursion.

Time Complexity

At every day we have roughly 2 choices:

```
O(2^n)
```

Space Complexity

Recursion stack:

```
O(n)
```

C++ Code

C++

```cpp
class Solution {
public:

    int solve(int idx, bool buy, vector<int>& prices) {

        if(idx == prices.size())
            return 0;

        if(buy) {

            int take =
                -prices[idx] +
                solve(idx + 1, false, prices);

            int skip =
                solve(idx + 1, true, prices);

            return max(take, skip);
        }
        else {

            int sell =
                prices[idx] +
                solve(idx + 1, true, prices);

            int hold =
                solve(idx + 1, false, prices);

            return max(sell, hold);
        }
    }

    int maxProfit(vector<int>& prices) {
        return solve(0, true, prices);
    }
};
```

3. Better Approach

Idea (DP Memoization)

The brute force repeatedly calculates the same states.

State:

```
(day, canBuy)
```

Store answers in DP.

Time Complexity

States:

```
n * 2
```

Therefore:

```
O(n)
```

Space Complexity

```
O(n)
```

for DP + recursion stack.

C++ Code

```C++
```

```cpp
class Solution {
public:

    vector<vector<int>> dp;

    int solve(int idx, int buy, vector<int>& prices) {

        if(idx == prices.size())
            return 0;

        if(dp[idx][buy] != -1)
            return dp[idx][buy];

        if(buy) {

            return dp[idx][buy] =
                max(
                    -prices[idx] + solve(idx + 1, 0, prices),
                    solve(idx + 1, 1, prices)
                );
        }

        return dp[idx][buy] =
            max(
                prices[idx] + solve(idx + 1, 1, prices),
                solve(idx + 1, 0, prices)
            );
    }

    int maxProfit(vector<int>& prices) {

        int n = prices.size();

        dp.assign(n, vector<int>(2, -1));

        return solve(0, 1, prices);
    }
};
```

---

4. Optimal Approach

Core Observation

Since unlimited transactions are allowed:

Whenever price increases, take that profit.

Example:

```
1 -> 5
```

Profit:

```
4
```

Can be viewed as:

```
(2-1) + (3-2) + (4-3) + (5-4)
```

Same result.

Key Greedy Observation

For every adjacent pair:

```cpp
prices[i] > prices[i-1]
```

add:

```cpp
prices[i] - prices[i-1]
```

to profit.

This captures every profitable rise.

Idea

Traverse array once.

If:

```cpp
prices[i] > prices[i-1]
```

add the difference.

Time Complexity

```
O(n)
```

Space Complexity

```
O(1)
```

C++ Code

```C++
class Solution {
public:
    int maxProfit(vector<int>& prices) {

        int profit = 0;

        for(int i = 1; i < prices.size(); i++) {

            if(prices[i] > prices[i - 1]) {
                profit += prices[i] - prices[i - 1];
            }
        }

        return profit;
    }
};
```

---

5. Dry Run

Input:

```
prices = [7,1,5,3,6,4]
```

Day 1

```
1 - 7 = -6
```

Ignore.

Profit:

```
0
```

Day 2

```
5 > 1
```

Add:

```
5 - 1 = 4
```

Profit:

```
4
```

Day 3

```
3 < 5
```

Ignore.

Profit:

```
4
```

Day 4

```
6 > 3
```

Add:

```
6 - 3 = 3
```

Profit:

```
7
```

Day 5

```
4 < 6
```

Ignore.

Final:

```
7
```

What Beginners Usually Miss

Observation 1

You do NOT need to find local minima and maxima explicitly.

This:

```C++
profit += max(0, prices[i] - prices[i-1]);
```

automatically captures every transaction.

Observation 2

For a sequence:

```
1 2 3 4 5
```

Profit:

```
5 - 1 = 4
```

and

```
(2-1)+(3-2)+(4-3)+(5-4)=4
```

Both are identical.

Observation 3 (The Trick)

Since transactions are unlimited:

```
Take every positive increase.
```

This greedy observation leads to the optimal solution.

Final Complexity

```
Time  : O(n)
Space : O(1)
```

Sources

Majority Element II

Majority Element II (LeetCode 229)

1. Problem Explanation (Simple English)

What is given?

An integer array `nums`.

Example:

```
nums = [3,2,3]
```

What do we need to find?

Return all elements that appear **more than ⌊n/3⌋ times**.

Example:

```
n = 3
n/3 = 1
```

Element `3` appears 2 times.

Answer:

```
[3]
```

Important Observation

There can be **at most 2 elements** occurring more than `n/3` times.

Why?

Suppose there were 3 such elements:

```
count(a) > n/3
count(b) > n/3
count(c) > n/3
```

Then:

```
count(a)+count(b)+count(c) > n
```

which is impossible.

This observation leads to the optimal solution.

---

2. Brute Force Approach

Idea

For every element:

- Count its occurrences by traversing the whole array.
- If count > n/3, add it to answer.
- Avoid duplicates.

Time Complexity

For each element:

```
O(n)
```

For all elements:

```
O(n²)
```

Space Complexity

```
O(1)
```

(ignoring answer array)

C++ Code

```C++
```

```cpp
class Solution {
public:
    vector<int> majorityElement(vector<int>& nums) {

        int n = nums.size();

        vector<int> ans;

        for(int i = 0; i < n; i++) {

            bool alreadyPresent = false;

            for(int x : ans) {
                if(x == nums[i]) {
                    alreadyPresent = true;
                    break;
                }
            }

            if(alreadyPresent) continue;

            int cnt = 0;

            for(int j = 0; j < n; j++) {
                if(nums[j] == nums[i])
                    cnt++;
            }

            if(cnt > n / 3)
                ans.push_back(nums[i]);
        }

        return ans;
    }
};
```

---

3. Better Approach

Idea (HashMap Frequency Count)

Step 1

Store frequency of every number.

C++

```cpp
unordered_map<int,int> mp;
```

Example:

```
[1,1,1,3,3,2,2,2]

1 -> 3
2 -> 3
3 -> 2
```

Step 2

Traverse hashmap.

If:

```C++
freq > n/3
```

add to answer.

Time Complexity

Building hashmap:

```
O(n)
```

Traversal:

```
O(n)
```

Overall:

```
O(n)
```

Space Complexity

```
O(n)
```

C++ Code

C++

```cpp
class Solution {
public:
    vector<int> majorityElement(vector<int>& nums) {

        unordered_map<int,int> mp;

        for(int x : nums)
            mp[x]++;

        vector<int> ans;

        int limit = nums.size() / 3;

        for(auto &it : mp) {

            if(it.second > limit)
                ans.push_back(it.first);
        }

        return ans;
    }
};
```

---

4. Optimal Approach

Core Observation

Since at most **2 elements** can appear more than `n/3` times,

we only need to keep track of:

```
candidate1
candidate2
```

This is the extended version of **Boyer-Moore Voting Algorithm**.

Idea

Maintain:

C++

```
candidate1
candidate2

count1
count2
```

Case 1

Current number equals candidate1

C++

```
count1++;
```

Case 2

Current number equals candidate2

C++

```
count2++;
```

Case 3

count1 == 0

C++

```
candidate1 = num;
count1 = 1;
```

Case 4

count2 == 0

C++

```
candidate2 = num;
count2 = 1;
```

Case 5

Different from both candidates

```C++
count1--;
count2--;
```

This effectively cancels three distinct elements.

Important

After first pass:

```
candidate1
candidate2
```

are only potential answers.

We must verify them in a second pass.

Time Complexity

First pass:

```
O(n)
```

Verification:

```
O(n)
```

Overall:

```
O(n)
```

Space Complexity

```
O(1)
```

C++ Code

C++

```cpp
class Solution {
public:
    vector<int> majorityElement(vector<int>& nums) {

        int candidate1 = 0;
        int candidate2 = 0;

        int count1 = 0;
        int count2 = 0;

        for(int num : nums) {

            if(num == candidate1) {
                count1++;
            }
            else if(num == candidate2) {
                count2++;
            }
            else if(count1 == 0) {
                candidate1 = num;
                count1 = 1;
            }
            else if(count2 == 0) {
                candidate2 = num;
                count2 = 1;
            }
            else {
                count1--;
                count2--;
            }
        }

        count1 = 0;
        count2 = 0;

        for(int num : nums) {

            if(num == candidate1)
                count1++;

            else if(num == candidate2)
                count2++;
        }

        vector<int> ans;

        int limit = nums.size() / 3;

        if(count1 > limit)
            ans.push_back(candidate1);

        if(count2 > limit)
            ans.push_back(candidate2);
```

```
        return ans;
    }
};
```

---

5. Dry Run

Input:

```
nums = [1,1,1,3,3,2,2,2]
```

Start

```
candidate1 = ?
candidate2 = ?

count1 = 0
count2 = 0
```

---

num = 1

```
candidate1 = 1
count1 = 1
```

---

num = 1

```
count1 = 2
```

---

num = 1

```
count1 = 3
```

---

num = 3

```
candidate2 = 3
count2 = 1
```

---

num = 3

```
count2 = 2
```

---

num = 2

Different from both candidates.

```
count1--
count2--
```

```
count1 = 2
count2 = 1
```

---

num = 2

```
count1 = 1
count2 = 0
```

---

num = 2

```
candidate2 = 2
count2 = 1
```

Candidates after first pass:

```
candidate1 = 1
candidate2 = 2
```

Verification:

```
1 occurs 3 times
2 occurs 3 times
```

Both:

```
3 > 8/3
```

Answer:

```
[1,2]
```

What Beginners Usually Miss

Trick 1

For `n/2` majority:

```
Only 1 candidate needed
```

For `n/3` majority:

```
2 candidates needed
```

For `n/k` majority:

```
k-1 candidates needed
```

Trick 2

The first pass does **not** guarantee the candidates are valid.

Always perform the second verification pass.

Trick 3 (Most Important)

When:

```
C++
```

```
count1--;
count2--;
```

you are effectively removing one occurrence of three different elements.

This cancellation is the heart of the Boyer-Moore Voting Algorithm.

Final Complexity

```
Time  : O(n)
Space : O(1)
```

Sources

Subarray Sum Equals K

Subarray Sum Equals K (LeetCode 560)

1. Problem Explanation (Simple English)

What is given?

An integer array `nums` and an integer `k`.

Example:

```
nums = [1,1,1]
k = 2
```

What do we need to find?

Count the number of **continuous subarrays** whose sum equals `k`.

Example:

```
[1,1] -> sum = 2
[1,1] -> sum = 2
```

Answer:

```
2
```

Constraint Intuition

- Array may contain:
  - Positive numbers
  - Negative numbers
  - Zeroes

Because negative numbers exist, **Sliding Window does NOT work**.

We need Prefix Sum + HashMap.

---

2. Brute Force Approach

Idea

Generate every possible subarray.

For each subarray:

- Compute its sum.
- If sum == k, increase answer.

Time Complexity

Number of subarrays:

```
O(n²)
```

Computing sum of each subarray:

```
O(n)
```

Total:

```
O(n³)
```

Space Complexity

```
O(1)
```

C++ Code

```C++
class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {

        int n = nums.size();
        int ans = 0;

        for(int i = 0; i < n; i++) {

            for(int j = i; j < n; j++) {

                int sum = 0;

                for(int p = i; p <= j; p++) {
                    sum += nums[p];
                }

                if(sum == k)
                    ans++;
            }
        }

        return ans;
    }
};
```

---

3. Better Approach

Idea (Prefix Sum Array)

Observation

If we know:

```
prefix[j]
```

and

```
prefix[i-1]
```

then:

```
sum(i...j)
=
prefix[j] - prefix[i-1]
```

Build prefix sum first.

Then check every subarray sum in O(1).

Time Complexity

Build prefix:

```
O(n)
```

Check all subarrays:

```
O(n²)
```

Total:

```
O(n²)
```

Space Complexity

```
O(n)
```

C++ Code

```
</> C++
```

```cpp
class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {

        int n = nums.size();

        vector<int> pref(n);

        pref[0] = nums[0];

        for(int i = 1; i < n; i++) {
            pref[i] = pref[i - 1] + nums[i];
        }

        int ans = 0;

        for(int i = 0; i < n; i++) {

            for(int j = i; j < n; j++) {

                int sum = pref[j];

                if(i > 0)
                    sum -= pref[i - 1];

                if(sum == k)
                    ans++;
            }
        }

        return ans;
    }
};
```

---

4. Optimal Approach

Core Observation

Let:

```
prefixSum = sum from index 0 to current index
```

Suppose:

```
currentPrefix = 10
k = 4
```

Then we need a previous prefix sum:

```
10 - previousPrefix = 4
```

So:

```
previousPrefix = 6
```

If prefix sum `6` occurred earlier, then a subarray with sum `k` exists.

Key Formula

For current prefix sum:

```
prefixSum
```

Need:

```
prefixSum - k
```

If it appeared before:

```
answer += frequency(prefixSum - k)
```

Idea

Maintain:

```cpp
unordered_map<int,int> mp;
```

where:

```
mp[x]
=
how many times prefix sum x has appeared
```

Initialize:

```cpp
mp[0] = 1;
```

This handles subarrays starting from index 0.

Time Complexity

Single traversal:

```
O(n)
```

Space Complexity

```
O(n)
```

C++ Code

C++

```cpp
class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {

        unordered_map<int,int> mp;

        mp[0] = 1;

        int prefixSum = 0;
        int ans = 0;

        for(int num : nums) {

            prefixSum += num;

            if(mp.count(prefixSum - k)) {
                ans += mp[prefixSum - k];
            }

            mp[prefixSum]++;
        }

        return ans;
    }
};
```

---

5. Dry Run

Input:

```
nums = [1,1,1]
k = 2
```

Initial:

```
mp = {0 : 1}

prefixSum = 0
ans = 0
```

Element = 1

```
prefixSum = 1
```

Need:

```
1 - 2 = -1
```

Not found.

Store:

```
mp[1]++
```

Map:

```
{0:1, 1:1}
```

Element = 1

```
prefixSum = 2
```

Need:

```
2 - 2 = 0
```

Found once.

```
ans += 1
```

Answer:

```
1
```

Store:

```
mp[2]++
```

Element = 1

```
prefixSum = 3
```

Need:

```
3 - 2 = 1
```

Found once.

```
ans += 1
```

Answer:

```
2
```

Store:

```
mp[3]++
```

Final Answer:

```
2
```

What Beginners Usually Miss

Observation 1

This is **not a Sliding Window problem**.

Example:

```
nums = [2,-1,2]
k = 3
```

Negative numbers break sliding window logic.

Observation 2

The most important equation:

```
currentPrefix - previousPrefix = k
```

Rearrange:

```
previousPrefix = currentPrefix - k
```

This is exactly what we search in the hashmap.

Observation 3

Always initialize:

```C++
mp[0] = 1;
```

Otherwise subarrays starting from index `0` will be missed.

Example:

```
nums = [3]
k = 3
```

Without:

```C++
mp[0] = 1;
```

the answer becomes incorrect.

Final Complexity

```
Time  : O(n)
Space : O(n)
```

This Prefix Sum + HashMap solution is the standard optimal solution for **Subarray Sum Equals K**.

Sources

First Missing Positive

First Missing Positive (LeetCode 41)

1. Problem Explanation (Simple English)

What is given?

An unsorted integer array `nums`.

Example:

```
nums = [1,2,0]
```

What do we need to find?

Find the **smallest missing positive integer**.

Example:

```
[1,2,0]
```

Positive numbers present:

```
1,2
```

Smallest missing positive:

```
3
```

More Examples

```
nums = [3,4,-1,1]

Positive numbers:
1,3,4

Answer = 2
```

```
nums = [7,8,9,11,12]

Answer = 1
```

Constraint Intuition

The problem asks for:

```
Time  : O(n)
Space : O(1)
```

Sorting (`O(n log n)`) is not optimal.

The trick is to use the array itself as a hash table.

---

2. Brute Force Approach

Idea

Start from:

```
1, 2, 3, 4, ...
```

For each number, scan the entire array to check whether it exists.

The first missing one is the answer.

Time Complexity

For every candidate:

```
O(n)
```

Search in array:

```
O(n)
```

Overall:

```
O(n²)
```

Space Complexity

```
O(1)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    int firstMissingPositive(vector<int>& nums) {

        int n = nums.size();

        for(int target = 1; target <= n + 1; target++) {

            bool found = false;

            for(int x : nums) {

                if(x == target) {
                    found = true;
                    break;
                }
            }

            if(!found)
                return target;
        }

        return n + 1;
    }
};
```

---

3. Better Approach

Idea (Sorting)

Observation

After sorting:

```
[3,4,-1,1]
```

becomes

```
[-1,1,3,4]
```

Now track the smallest positive expected value.

Steps

Start:

```
need = 1
```

Traverse sorted array:

- If current value equals `need`
  - increment `need`
- Ignore duplicates.
- Ignore negatives.

At the end:

```
need
```

is the answer.

Time Complexity

Sorting:

```
O(n log n)
```

Traversal:

```
O(n)
```

Total:

```
O(n log n)
```

Space Complexity

```
O(1)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    int firstMissingPositive(vector<int>& nums) {

        sort(nums.begin(), nums.end());

        int need = 1;

        for(int x : nums) {

            if(x == need)
                need++;

            else if(x < need)
                continue;
        }

        return need;
    }
};
```

---

4. Optimal Approach

Core Observation

For an array of size `n`:

The answer must lie in:

```
[1 ... n+1]
```

Why?

Example:

```
n = 5
```

If:

```
1,2,3,4,5
```

are all present,

answer is:

```
6
```

Otherwise answer is inside:

```
1..5
```

Key Idea

Place every number `x` at index:

```
x - 1
```

because:

```
1 should be at index 0
2 should be at index 1
3 should be at index 2
...
```

This is called **Cyclic Sort**.

Steps

Step 1

For each index:

If:

```C++
nums[i] > 0
nums[i] <= n
```

and

```C++
nums[i] != nums[nums[i]-1]
```

swap them.

Step 2

After rearrangement:

Ideal array:

```
index : 0 1 2 3

value : 1 2 3 4
```

Now scan.

First index where:

```C++
nums[i] != i+1
```

Answer:

```C++
i+1
```

Step 3

If all positions are correct:

Return:

```C++
n+1
```

Time Complexity

Each element is moved at most once/twice.

```
O(n)
```

Space Complexity

```
O(1)
```

C++ Code

C++

```cpp
class Solution {
public:
    int firstMissingPositive(vector<int>& nums) {

        int n = nums.size();

        int i = 0;

        while(i < n) {

            int correct = nums[i] - 1;

            if(nums[i] > 0 &&
               nums[i] <= n &&
               nums[i] != nums[correct]) {

                swap(nums[i], nums[correct]);
            }
            else {
                i++;
            }
        }

        for(int i = 0; i < n; i++) {

            if(nums[i] != i + 1)
                return i + 1;
        }

        return n + 1;
    }
};
```

---

5. Dry Run

Input:

```
nums = [3,4,-1,1]
```

Initial

```
[3,4,-1,1]
```

i = 0

```
nums[0] = 3

correct = 2
```

Swap:

```
[-1,4,3,1]
```

i = 0

```
-1
```

Invalid.

Move ahead.

i = 1

```
4

correct = 3
```

Swap:

```
[-1,1,3,4]
```

i = 1

```
1
```

```
correct = 0
```

Swap:

```
[1,-1,3,4]
```

Final Arrangement

```
[1,-1,3,4]
```

Check Positions

Index 0:

```
1 == 1
```

Good.

Index 1:

```
-1 != 2
```

Therefore:

```
Answer = 2
```

What Beginners Usually Miss

Observation 1

The answer is never larger than:

```
n + 1
```

This is the key mathematical observation.

Observation 2

Ignore:

```
negative numbers
0
numbers > n
```

They can never affect the answer.

Observation 3 (Most Important)

This condition:

```cpp
nums[i] != nums[nums[i]-1]
```

prevents infinite swapping when duplicates exist.

Example:

```
[1,1]
```

Without this condition:

```cpp
swap(nums[1], nums[0]);
```

would happen forever.

Final Complexity

```
Time  : O(n)
Space : O(1)
```

This Cyclic Sort approach is the expected optimal interview solution.

Sources

Two Sum II Input Array Is Sorted

Two Sum II - Input Array Is Sorted (LeetCode 167)

1. Problem Explanation (Simple English)

What is given?

A **sorted** array `numbers` and an integer `target`.

Example:

```
numbers = [2,7,11,15]
target = 9
```

What do we need to find?

Find two numbers whose sum equals `target`.

Return their **1-based indices**.

Example:

```
2 + 7 = 9
```

Indices:

```
1 and 2
```

Answer:

```
[1,2]
```

Important

- Array is already sorted.
- Exactly one valid answer exists.
- Cannot use the same element twice.

---

2. Brute Force Approach

Idea

Try every pair.

For each index `i`:

- Check every index `j > i`.
- If:

```C++
numbers[i] + numbers[j] == target
```

return answer.

Time Complexity

Two nested loops:

```
O(n²)
```

Space Complexity

```
O(1)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {

        int n = numbers.size();

        for(int i = 0; i < n; i++) {

            for(int j = i + 1; j < n; j++) {

                if(numbers[i] + numbers[j] == target) {
                    return {i + 1, j + 1};
                }
            }
        }

        return {};
    }
};
```

---

3. Better Approach

Idea (Binary Search)

Observation

For every element:

```cpp
numbers[i]
```

we need:

```cpp
target - numbers[i]
```

Since the array is sorted,

we can binary search for the required value.

Steps

For every index:

```C++
need = target - numbers[i]
```

Binary search in:

```
[i+1 ... n-1]
```

Time Complexity

For every element:

```
O(log n)
```

Total:

```
O(n log n)
```

Space Complexity

```
O(1)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {

        int n = numbers.size();

        for(int i = 0; i < n; i++) {

            int need = target - numbers[i];

            int l = i + 1;
            int r = n - 1;

            while(l <= r) {

                int mid = l + (r - l) / 2;

                if(numbers[mid] == need) {
                    return {i + 1, mid + 1};
                }
                else if(numbers[mid] < need) {
                    l = mid + 1;
                }
                else {
                    r = mid - 1;
                }
            }
        }

        return {};
    }
};
```

---

4. Optimal Approach

Core Observation

The array is sorted.

Use two pointers:

```cpp
left = 0
right = n - 1
```

Case 1

If:

```cpp
numbers[left] + numbers[right] == target
```

Answer found.

Case 2

If sum is too small:

```cpp
sum < target
```

Increase left pointer.

Why?

Because moving right leftwards only decreases the sum.

Case 3

If sum is too large:

```cpp
sum > target
```

Decrease right pointer.

Why?

Because moving left rightwards only increases the sum.

Idea

Keep shrinking the search space.

At each step one pointer moves.

Time Complexity

Each pointer moves at most `n` times.

```
O(n)
```

Space Complexity

```
O(1)
```

C++ Code

```C++
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {

        int left = 0;
        int right = numbers.size() - 1;

        while(left < right) {

            int sum = numbers[left] + numbers[right];

            if(sum == target) {
                return {left + 1, right + 1};
            }
            else if(sum < target) {
                left++;
            }
            else {
                right--;
            }
        }

        return {};
    }
};
```

---

5. Dry Run

Input:

```
numbers = [2,7,11,15]
target = 9
```

Initial

```
left = 0
right = 3
```

Values:

```
2 and 15
```

Sum:

```
17
```

Since:

```
17 > 9
```

Move right.

Step 2

```
left = 0
right = 2
```

Values:

```
2 and 11
```

Sum:

```
13
```

Still greater.

Move right.

Step 3

```
left = 0
right = 1
```

Values:

```
2 and 7
```

Sum:

```
9
```

Found.

Return:

```
[1,2]
```

What Beginners Usually Miss

Observation 1

This is different from **Two Sum I**.

For Two Sum I:

```
Use HashMap
```

because array is unsorted.

Observation 2

Since array is sorted:

```
HashMap is unnecessary.
```

Two pointers are better.

Observation 3

Why does Two Pointer work?

Because sorting creates a monotonic relationship:

```
Move left  -> sum increases
Move right -> sum decreases
```

This allows us to eliminate many pairs at once.

Final Complexity

Brute Force

```
Time  : O(n²)
Space : O(1)
```

Binary Search

```
Time  : O(n log n)
Space : O(1)
```

Optimal (Two Pointers)

```
Time  : O(n)
Space : O(1)
```

This is the expected interview and LeetCode optimal solution.

Sources

3Sum

3Sum (LeetCode 15)

1. Problem Explanation (Simple English)

What is given?

An integer array `nums`.

Example:

```
nums = [-1,0,1,2,-1,-4]
```

What do we need to find?

Find all unique triplets:

```
[a,b,c]
```

such that:

```
a + b + c = 0
```

Example

Input:

```
[-1,0,1,2,-1,-4]
```

Output:

```
[[-1,-1,2],
 [-1,0,1]]
```

Important

- Triplets must be unique.
- Order inside a triplet doesn't matter.
- No duplicate triplets in the answer.

Constraint Intuition

If we try all triplets:

```
O(n³)
```

This is too slow for large `n`.

---

2. Brute Force Approach

Idea

Try every possible triplet.

Steps

- Pick first element `i`
- Pick second element `j`
- Pick third element `k`
- If sum is 0, store it.
- Sort the triplet before inserting into a set to avoid duplicates.

Time Complexity

Three nested loops:

```
O(n³)
```

Space Complexity

```
O(number of triplets)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {

        int n = nums.size();

        set<vector<int>> st;

        for(int i = 0; i < n; i++) {

            for(int j = i + 1; j < n; j++) {

                for(int k = j + 1; k < n; k++) {

                    if(nums[i] + nums[j] + nums[k] == 0) {

                        vector<int> temp = {
                            nums[i],
                            nums[j],
                            nums[k]
                        };

                        sort(temp.begin(), temp.end());

                        st.insert(temp);
                    }
                }
            }
        }

        return vector<vector<int>>(st.begin(), st.end());
    }
};
```

---

3. Better Approach

Idea (HashSet)

Fix one element.

For remaining elements, solve a Two Sum problem.

Steps

For every index `i`:

Need:

```
nums[j] + nums[k] = -nums[i]
```

Use a hash set to find the required element.

Time Complexity

Outer loop:

```
O(n)
```

HashSet search:

```
O(n)
```

Total:

```
O(n²)
```

Space Complexity

```
O(n)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {

        int n = nums.size();

        set<vector<int>> ans;

        for(int i = 0; i < n; i++) {

            unordered_set<int> st;

            for(int j = i + 1; j < n; j++) {

                int need = -(nums[i] + nums[j]);

                if(st.count(need)) {

                    vector<int> temp = {
                        nums[i],
                        nums[j],
                        need
                    };

                    sort(temp.begin(), temp.end());

                    ans.insert(temp);
                }

                st.insert(nums[j]);
            }
        }

        return vector<vector<int>>(ans.begin(), ans.end());
    }
};
```

---

4. Optimal Approach

Core Observation

After sorting:

```
[-4,-1,-1,0,1,2]
```

For every fixed element:

```
nums[i]
```

we need:

```
nums[left] + nums[right] = -nums[i]
```

This becomes exactly a **Two Sum in Sorted Array** problem.

Idea

Step 1

Sort array.

```C++
sort(nums.begin(), nums.end());
```

Step 2

Fix first element.

```C++
for(i=0;i<n;i++)
```

Step 3

Use two pointers.

```C++
left = i + 1
right = n - 1
```

Check:

```C++
sum = nums[i] + nums[left] + nums[right]
```

Case 1

```
sum < 0
```

Need larger value.

```C++
left++;
```

Case 2

```
sum > 0
```

Need smaller value.

```C++
right--;
```

Case 3

```
sum == 0
```

Triplet found.

Store it.

Skip duplicates.

Handling Duplicates

Skip duplicate first elements

```C++
if(i > 0 && nums[i] == nums[i-1])
    continue;
```

Skip duplicate left values

After finding a triplet:

```cpp
while(left < right &&
      nums[left] == nums[left+1])
    left++;
```

Skip duplicate right values

```cpp
while(left < right &&
      nums[right] == nums[right-1])
    right--;
```

Time Complexity

Sorting:

```
O(n log n)
```

Two pointer for every index:

```
O(n²)
```

Overall:

```
O(n²)
```

Space Complexity

```
O(1)
```

Ignoring output array.

C++ Code

C++

```cpp
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {

        sort(nums.begin(), nums.end());

        int n = nums.size();

        vector<vector<int>> ans;

        for(int i = 0; i < n; i++) {

            if(i > 0 && nums[i] == nums[i - 1])
                continue;

            int left = i + 1;
            int right = n - 1;

            while(left < right) {

                long long sum =
                    (long long)nums[i]
                    + nums[left]
                    + nums[right];

                if(sum < 0) {
                    left++;
                }
                else if(sum > 0) {
                    right--;
                }
                else {

                    ans.push_back({
                        nums[i],
                        nums[left],
                        nums[right]
                    });

                    while(left < right &&
                          nums[left] == nums[left + 1])
                        left++;

                    while(left < right &&
                          nums[right] == nums[right - 1])
                        right--;

                    left++;
                    right--;
                }
            }
        }
```

```
        return ans;
    }
};
```

---

5. Dry Run

Input:

```
[-1,0,1,2,-1,-4]
```

After sorting:

```
[-4,-1,-1,0,1,2]
```

---

i = 0

```
nums[i] = -4
```

Need:

```
left + right = 4
```

No valid pair.

---

i = 1

```
nums[i] = -1
```

Pointers:

```
left = 2
right = 5
```

Sum:

```
-1 + (-1) + 2 = 0
```

Triplet:

```
[-1,-1,2]
```

Store.

Move pointers.

Now:

```
left = 3
right = 4
```

Sum:

```
-1 + 0 + 1 = 0
```

Triplet:

```
[-1,0,1]
```

Store.

---

i = 2

Duplicate:

```
nums[2] == nums[1]
```

Skip.

Final Answer

```
[
 [-1,-1,2],
 [-1,0,1]
]
```

What Beginners Usually Miss

Trick 1

Sorting is mandatory.

Without sorting, two pointers cannot be used.

Trick 2

Always skip duplicate `i`.

```C++
if(i > 0 && nums[i] == nums[i-1])
    continue;
```

Otherwise duplicate triplets appear.

Trick 3

After finding a valid triplet:

```C++
while(nums[left] == nums[left+1])
    left++;

while(nums[right] == nums[right-1])
    right--;
```

This is the most commonly missed part.

Final Complexity

Brute Force

```
Time  : O(n³)
Space : O(k)
```

Better (HashSet)

```
Time  : O(n²)
Space : O(n)
```

Optimal (Sorting + Two Pointers)

```
Time  : O(n²)
Space : O(1)
```

This is the expected interview and LeetCode optimal solution.

Sources

4Sum

4Sum (LeetCode 18)

1. Problem Explanation (Simple English)

What is given?

An integer array `nums` and an integer `target`.

Example:

```
nums = [1,0,-1,0,-2,2]
target = 0
```

What do we need to find?

Find all unique quadruplets:

```
[a,b,c,d]
```

such that:

```
a + b + c + d = target
```

Example

Output:

```
[
 [-2,-1,1,2],
 [-2,0,0,2],
 [-1,0,0,1]
]
```

Important

- No duplicate quadruplets.
- Order inside quadruplet doesn't matter.
- Return all valid quadruplets.

---

2. Brute Force Approach

Idea

Try every possible quadruplet.

Steps

- Choose first index `i`
- Choose second index `j`
- Choose third index `k`
- Choose fourth index `l`
- If sum equals target, store it.

Use a set to remove duplicates.

Time Complexity

Four nested loops:

```
O(n⁴)
```

Space Complexity

```
O(number of answers)
```

C++ Code

C++

```cpp
class Solution {
public:
    vector<vector<int>> fourSum(vector<int>& nums, int target) {

        int n = nums.size();

        set<vector<int>> st;

        for(int i = 0; i < n; i++) {

            for(int j = i + 1; j < n; j++) {

                for(int k = j + 1; k < n; k++) {

                    for(int l = k + 1; l < n; l++) {

                        long long sum =
                            (long long)nums[i] +
                            nums[j] +
                            nums[k] +
                            nums[l];

                        if(sum == target) {

                            vector<int> temp = {
                                nums[i],
                                nums[j],
                                nums[k],
                                nums[l]
                            };

                            sort(temp.begin(), temp.end());

                            st.insert(temp);
                        }
                    }
                }
            }
        }

        return vector<vector<int>>(st.begin(), st.end());
    }
};
```

---

3. Better Approach

Idea (HashSet)

Fix two elements.

For the remaining two elements, solve a Two Sum problem using a hash set.

Steps

- Fix `i`
- Fix `j`
- Find:

```
nums[k] + nums[l]
=
target - nums[i] - nums[j]
```

using HashSet.

Store quadruplets in a set to avoid duplicates.

Time Complexity

```
O(n³)
```

Space Complexity

```
O(n)
```

C++ Code

```
</> C++
```

```cpp
class Solution {
public:
    vector<vector<int>> fourSum(vector<int>& nums, int target) {

        int n = nums.size();

        set<vector<int>> ans;

        for(int i = 0; i < n; i++) {

            for(int j = i + 1; j < n; j++) {

                unordered_set<long long> st;

                for(int k = j + 1; k < n; k++) {

                    long long need =
                        (long long)target
                        - nums[i]
                        - nums[j]
                        - nums[k];

                    if(st.count(need)) {

                        vector<int> temp = {
                            nums[i],
                            nums[j],
                            nums[k],
                            (int)need
                        };

                        sort(temp.begin(), temp.end());

                        ans.insert(temp);
                    }

                    st.insert(nums[k]);
                }
            }
        }

        return vector<vector<int>>(ans.begin(), ans.end());
    }
};
```

---

4. Optimal Approach

Core Observation

This is exactly the extension of **3Sum**.

For 3Sum

```
Fix 1 element
+
Two Pointers
```

For 4Sum

```
Fix 2 elements
+
Two Pointers
```

Idea

Step 1

Sort the array.

```cpp
sort(nums.begin(), nums.end());
```

Step 2

Fix first element:

```cpp
i
```

Step 3

Fix second element:

```cpp
j
```

Step 4

Use Two Pointers for remaining two numbers.

```C++
left = j + 1;
right = n - 1;
```

Check:

```C++
sum =
nums[i]
+ nums[j]
+ nums[left]
+ nums[right];
```

Case 1

```
sum < target
```

Need larger value.

```C++
left++;
```

Case 2

```
sum > target
```

Need smaller value.

```C++
right--;
```

Case 3

```
sum == target
```

Store quadruplet.

Skip duplicates.

Move both pointers.

Handling Duplicates

Skip duplicate i

```C++
if(i > 0 && nums[i] == nums[i-1])
    continue;
```

Skip duplicate j

```C++
if(j > i+1 && nums[j] == nums[j-1])
    continue;
```

Skip duplicate left/right

After finding a quadruplet:

```C++
while(left < right &&
      nums[left] == nums[left+1])
    left++;

while(left < right &&
      nums[right] == nums[right-1])
    right--;
```

Time Complexity

Sorting:

```
O(n log n)
```

Two fixed loops:

```
O(n²)
```

Two pointers:

```
O(n)
```

Total:

```
O(n³)
```

Space Complexity

```
O(1)
```

Ignoring answer array.

C++ Code

```C++
```

```cpp
class Solution {
public:
    vector<vector<int>> fourSum(vector<int>& nums, int target) {

        sort(nums.begin(), nums.end());

        int n = nums.size();

        vector<vector<int>> ans;

        for(int i = 0; i < n; i++) {

            if(i > 0 && nums[i] == nums[i - 1])
                continue;

            for(int j = i + 1; j < n; j++) {

                if(j > i + 1 &&
                   nums[j] == nums[j - 1])
                    continue;

                int left = j + 1;
                int right = n - 1;

                while(left < right) {

                    long long sum =
                        (long long)nums[i]
                        + nums[j]
                        + nums[left]
                        + nums[right];

                    if(sum < target) {
                        left++;
                    }
                    else if(sum > target) {
                        right--;
                    }
                    else {

                        ans.push_back({
                            nums[i],
                            nums[j],
                            nums[left],
                            nums[right]
                        });

                        while(left < right &&
                              nums[left] == nums[left + 1])
                            left++;

                        while(left < right &&
                              nums[right] == nums[right - 1])
```

```
                        right--;

                    left++;
                    right--;
                }
            }
        }
    }

    return ans;
  }
};
```

---

5. Dry Run

Input:

```
nums = [1,0,-1,0,-2,2]
target = 0
```

After sorting:

```
[-2,-1,0,0,1,2]
```

---

i = 0

```
-2
```

j = 1

```
-1
```

Need:

```
3
```

Pointers:

```
left = 2
right = 5
```

Sum:

```
-2 + (-1) + 0 + 2 = -1
```

Move left.

Now:

```
-2 + (-1) + 1 + 2 = 0
```

Found:

```
[-2,-1,1,2]
```

---

j = 2

```
0
```

Pointers:

```
left = 3
right = 5
```

Sum:

```
-2 + 0 + 0 + 2 = 0
```

Found:

```
[-2,0,0,2]
```

---

i = 1

```
-1
```

j = 2

```
0
```

Sum:

```
-1 + 0 + 0 + 1 = 0
```

Found:

```
[-1,0,0,1]
```

Final Answer

```
[
 [-2,-1,1,2],
 [-2,0,0,2],
 [-1,0,0,1]
]
```

What Beginners Usually Miss

Trick 1

Always use:

```C++
long long sum
```

because values can be large.

Example:

```
1000000000
1000000000
1000000000
1000000000
```

can overflow `int`.

Trick 2

4Sum is simply:

```
3Sum + one extra fixed loop
```

Trick 3

Duplicate handling is the most important part.

Skip duplicates for:

```
i
j
left
right
```

otherwise repeated quadruplets appear.

Final Complexity

Brute Force

```
Time  : O(n⁴)
Space : O(k)
```

Better (HashSet)

```
Time  : O(n³)
Space : O(n)
```

Optimal (Sort + Two Pointers)

```
Time  : O(n³)
Space : O(1)
```

This is the expected interview and LeetCode optimal solution.

Sources

Rotate ArrayRotate Array

Rotate Array (LeetCode 189)

1. Problem Explanation (Simple English)

What is given?

An integer array `nums` and an integer `k`.

Example:

```
nums = [1,2,3,4,5,6,7]
k = 3
```

What do we need to do?

Rotate the array to the **right** by `k` steps.

Example:

```
Before:
[1,2,3,4,5,6,7]

After 1 rotation:
[7,1,2,3,4,5,6]

After 2 rotations:
[6,7,1,2,3,4,5]

After 3 rotations:
[5,6,7,1,2,3,4]
```

Output:

```
[5,6,7,1,2,3,4]
```

Constraint Intuition

- `n` can be large.
- Rotating one step at a time is inefficient.
- We need an `O(n)` solution.

---

2. Brute Force Approach

Idea

Rotate the array one step to the right exactly `k` times.

One Rotation

```
[1,2,3,4]

↓

[4,1,2,3]
```

Repeat this `k` times.

Time Complexity

One rotation:

```
O(n)
```

Perform `k` times:

```
O(n*k)
```

Space Complexity

```
O(1)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    void rotate(vector<int>& nums, int k) {

        int n = nums.size();

        while(k--) {

            int last = nums[n - 1];

            for(int i = n - 1; i > 0; i--) {
                nums[i] = nums[i - 1];
            }

            nums[0] = last;
        }
    }
};
```

---

3. Better Approach

Idea (Extra Array)

Observation

After rotation:

```
newIndex = (i + k) % n
```

Example:

```
nums = [1,2,3,4,5,6,7]
k = 3
```

For:

```
i = 0
```

```
newIndex = (0+3)%7 = 3
```

So:

```
1 goes to index 3
```

Steps

Create another array.

Place each element at its rotated position.

Copy back.

Time Complexity

```
O(n)
```

Space Complexity

```
O(n)
```

C++ Code

C++

```cpp
class Solution {
public:
    void rotate(vector<int>& nums, int k) {

        int n = nums.size();

        k %= n;

        vector<int> temp(n);

        for(int i = 0; i < n; i++) {
            temp[(i + k) % n] = nums[i];
        }

        nums = temp;
    }
};
```

---

4. Optimal Approach

Core Observation

Use the **Reversal Algorithm**.

Example:

```
nums = [1,2,3,4,5,6,7]
k = 3
```

Target:

```
[5,6,7,1,2,3,4]
```

Step 1

Reverse entire array.

```
[1,2,3,4,5,6,7]

↓

[7,6,5,4,3,2,1]
```

Step 2

Reverse first `k` elements.

```
[7,6,5]

↓

[5,6,7]
```

Array:

```
[5,6,7,4,3,2,1]
```

Step 3

Reverse remaining elements.

```
[4,3,2,1]

↓

[1,2,3,4]
```

Final:

```
[5,6,7,1,2,3,4]
```

Why Does This Work?

After reversing the entire array:

```
Last k elements
+
First n-k elements
```

come into the correct groups.

We only need to fix their internal order.

Time Complexity

Three reversals:

```
O(n)
```

Space Complexity

```
O(1)
```

C++ Code

```
</> C++
```

```cpp
class Solution {
public:
    void rotate(vector<int>& nums, int k) {

        int n = nums.size();

        k %= n;

        reverse(nums.begin(), nums.end());

        reverse(nums.begin(), nums.begin() + k);

        reverse(nums.begin() + k, nums.end());
    }
};
```

---

5. Dry Run

Input:

```
nums = [1,2,3,4,5,6,7]
k = 3
```

Step 1

Reverse whole array.

```
[7,6,5,4,3,2,1]
```

Step 2

Reverse first 3 elements.

```
[5,6,7,4,3,2,1]
```

Step 3

Reverse remaining part.

```
[5,6,7,1,2,3,4]
```

Final Answer

```
[5,6,7,1,2,3,4]
```

What Beginners Usually Miss

Observation 1

Always do:

```cpp
k %= n;
```

Example:

```
n = 7
k = 10
```

```
10 % 7 = 3
```

Rotating by 10 is same as rotating by 3.

Observation 2

The reversal algorithm is one of the most commonly asked array tricks.

Remember:

```
Reverse All
Reverse First k
Reverse Remaining
```

Observation 3

For left rotation by `k`:

```
Reverse First k
Reverse Remaining
Reverse Entire Array
```

Final Complexity

Brute Force

```
Time  : O(n*k)
Space : O(1)
```

Better

```
Time  : O(n)
Space : O(n)
```

Optimal (Reversal Algorithm)

```
Time  : O(n)
Space : O(1)
```

This is the expected interview and LeetCode optimal solution.

Sources

Container With Most Water Container With Most Water

Container With Most Water (LeetCode 11)

1. Problem Explanation (Simple English)

What is given?

An array `height`.

```
height[i]
```

represents the height of a vertical line at index `i`.

Example:

```
height = [1,8,6,2,5,4,8,3,7]
```

What do we need to find?

Choose two lines that form a container with the x-axis.

Return the **maximum amount of water** the container can hold.

How is area calculated?

If we choose indices:

```
i and j
```

Then:

```
width = j - i

height = min(height[i], height[j])

area = width * height
```

Example:

```
height = [1,8,6,2,5,4,8,3,7]

Choose:

8 and 7

width = 8 - 1 = 7

height = min(8,7) = 7

area = 7 * 7 = 49
```

Answer:

```
49
```

Constraint Intuition

Trying all pairs:

```
O(n²)
```

But we can do better using Two Pointers.

---

2. Brute Force Approach

Idea

Try every pair of lines.

For each pair:

```cpp
area =
(j - i)
*
min(height[i], height[j])
```

Keep maximum.

Time Complexity

Two nested loops:

```
O(n²)
```

Space Complexity

```
O(1)
```

C++ Code

C++

```cpp
class Solution {
public:
    int maxArea(vector<int>& height) {

        int n = height.size();

        int ans = 0;

        for(int i = 0; i < n; i++) {

            for(int j = i + 1; j < n; j++) {

                int area =
                    (j - i) *
                    min(height[i], height[j]);

                ans = max(ans, area);
            }
        }

        return ans;
    }
};
```

---

3. Better Approach

Idea

There is no meaningful intermediate approach better than `O(n²)` that is commonly used.

Most interviewers expect moving directly from brute force to the optimal two-pointer solution.

Time Complexity

```
O(n²)
```

Space Complexity

```
O(1)
```

C++ Code

Same as brute force.

---

4. Optimal Approach

Core Observation

Area depends on:

```
width × smaller_height
```

Initially:

```
left = 0
right = n-1
```

This gives the maximum possible width.

Key Question

Which pointer should we move?

Suppose:

```
height[left] < height[right]
```

Current area:

```
(right-left)
*
height[left]
```

The smaller height is limiting the area.

Moving the taller line cannot help because:

```
width decreases
smaller height remains same or worse
```

So we must move the smaller height.

Important Trick

If

```cpp
height[left] < height[right]
```

Move:

```cpp
left++
```

Else

Move:

```cpp
right--
```

Why does this work?

The only way to get a larger area is:

```
increase the limiting height
```

The limiting height is always the smaller line.

Time Complexity

Each pointer moves at most `n` times.

```
O(n)
```

Space Complexity

```
O(1)
```

C++ Code

C++

```cpp
class Solution {
public:
    int maxArea(vector<int>& height) {

        int left = 0;
        int right = height.size() - 1;

        int ans = 0;

        while(left < right) {

            int area =
                (right - left) *
                min(height[left], height[right]);

            ans = max(ans, area);

            if(height[left] < height[right]) {
                left++;
            }
            else {
                right--;
            }
        }

        return ans;
    }
};
```

5. Dry Run

Input:

```
height =
[1,8,6,2,5,4,8,3,7]
```

Initial

```
left = 0
right = 8
```

Area:

```
width = 8

height = min(1,7) = 1

area = 8
```

Answer:

```
8
```

Since:

```
1 < 7
```

Move left.

Step 2

```
left = 1
right = 8
```

Area:

```
width = 7

height = min(8,7)=7

area = 49
```

Answer:

```
49
```

Since:

```
8 > 7
```

Move right.

Step 3

```
left = 1
right = 7
```

Area:

```
6 * min(8,3)
=
18
```

No improvement.

Move right.

Continue...

Maximum remains:

```
49
```

What Beginners Usually Miss

Observation 1

Area is determined by:

```
min(height[left], height[right])
```

not the larger height.

Observation 2

Never move the taller line.

Example:

```
2 ..... 100
```

Moving `100` inward only decreases width.

The limiting height is still `2`.

Observation 3 (Most Important)

The greedy decision is:

```C++
Move the smaller height.
```

This is the entire trick behind the `O(n)` solution.

Final Complexity

Brute Force

```
Time  : O(n²)
Space : O(1)
```

Optimal (Two Pointers)

```
Time  : O(n)
Space : O(1)
```

This is the expected interview and LeetCode optimal solution.

Sources

Boats to Save People

Boats to Save People (LeetCode 881)

1. Problem Explanation (Simple English)

What is given?

- An array `people` where:

```
people[i]
```

is the weight of the `i-th` person.

- A boat weight limit `limit`.

Example:

```
people = [1,2]
limit = 3
```

Boat Rules

Each boat can carry:

```
At most 2 people
```

and

```
Total weight <= limit
```

What do we need to find?

Return the **minimum number of boats** needed to rescue everyone.

Example:

```
people = [1,2]
limit = 3
```

One boat can take both.

Answer:

```
1
```

---

2. Brute Force Approach

Idea

For every heaviest person:

- Search the entire remaining array.
- Find the heaviest person that can fit with them.
- Mark both as used.
- Otherwise send the heavy person alone.

Time Complexity

For every person:

```
O(n)
```

Search partner:

```
O(n)
```

Overall:

```
O(n²)
```

Space Complexity

```
O(n)
```

(visited array)

C++ Code

```C++
```

```cpp
class Solution {
public:
    int numRescueBoats(vector<int>& people, int limit) {

        int n = people.size();

        vector<bool> vis(n, false);

        int boats = 0;

        for(int i = n - 1; i >= 0; i--) {

            if(vis[i]) continue;

            vis[i] = true;

            int partner = -1;

            for(int j = i - 1; j >= 0; j--) {

                if(!vis[j] &&
                   people[i] + people[j] <= limit) {

                    partner = j;
                    break;
                }
            }

            if(partner != -1)
                vis[partner] = true;

            boats++;
        }

        return boats;
    }
};
```

---

3. Better Approach

Idea

Sort the array first.

Then for every heaviest person:

- Binary search the best partner.
- Mark them used.

This improves searching.

Time Complexity

Sorting:

```
O(n log n)
```

For each person:

```
O(log n)
```

Overall:

```
O(n log n)
```

Space Complexity

```
O(n)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    int numRescueBoats(vector<int>& people, int limit) {

        sort(people.begin(), people.end());

        vector<bool> vis(people.size(), false);

        int boats = 0;

        for(int i = people.size() - 1; i >= 0; i--) {

            if(vis[i]) continue;

            vis[i] = true;

            int l = 0;
            int r = i - 1;
            int pos = -1;

            while(l <= r) {

                int mid = l + (r - l) / 2;

                if(!vis[mid] &&
                   people[mid] + people[i] <= limit) {

                    pos = mid;
                    l = mid + 1;
                }
                else {
                    r = mid - 1;
                }
            }

            if(pos != -1)
                vis[pos] = true;

            boats++;
        }

        return boats;
    }
};
```

---

4. Optimal Approach

Core Observation

The heaviest person must go on a boat.

Suppose:

```
people = [1,2,2,3]
limit = 3
```

Person:

```
3
```

cannot pair with anyone.

So:

```
3 must take a boat.
```

Greedy Idea

Sort the array.

Use two pointers:

```cpp
left = lightest person
right = heaviest person
```

Case 1

If:

```cpp
people[left] + people[right] <= limit
```

Then pair them.

```cpp
left++;
right--;
boats++;
```

Case 2

Otherwise:

```C++
people[left] + people[right] > limit
```

Then the heaviest person cannot pair with anyone.

Send them alone.

```C++
right--;
boats++;
```

Why is this Greedy Correct?

If the lightest person cannot fit with the heaviest:

```
lightest + heaviest > limit
```

then nobody can fit with the heaviest.

Because everyone else is heavier than the lightest.

Therefore:

```
heaviest must go alone.
```

This observation makes the greedy optimal.

Time Complexity

Sorting:

```
O(n log n)
```

Two pointers:

```
O(n)
```

Overall:

```
O(n log n)
```

Space Complexity

```
O(1)
```

(ignoring sorting implementation)

C++ Code

C++

```cpp
class Solution {
public:
    int numRescueBoats(vector<int>& people, int limit) {

        sort(people.begin(), people.end());

        int left = 0;
        int right = people.size() - 1;

        int boats = 0;

        while(left <= right) {

            if(people[left] + people[right] <= limit) {
                left++;
                right--;
            }
            else {
                right--;
            }

            boats++;
        }

        return boats;
    }
};
```

5. Dry Run

Input:

```
people = [3,2,2,1]
limit = 3
```

After sorting:

```
[1,2,2,3]
```

Step 1

```
left = 0
right = 3
```

Check:

```
1 + 3 = 4
```

Too large.

So:

```
3 goes alone
```

Boats:

```
1
```

Move:

```
right = 2
```

Step 2

```
left = 0
right = 2
```

Check:

```
1 + 2 = 3
```

Fits.

Pair them.

Boats:

```
2
```

Move:

```
left = 1
right = 1
```

Step 3

Only:

```
2
```

left.

Needs one boat.

Boats:

```
3
```

Final Answer

```
3
```

What Beginners Usually Miss

Observation 1

Always sort first.

Without sorting, the greedy pairing is impossible.

Observation 2

The key proof:

If:

```
lightest + heaviest > limit
```

then:

```
heaviest cannot pair with anyone.
```

This is the entire greedy argument.

Observation 3

Use:

```C++
while(left <= right)
```

not:

```C++
while(left < right)
```

because one person may remain at the end and still need a boat.

Final Complexity

Brute Force

```
Time  : O(n²)
Space : O(n)
```

Optimal (Sort + Two Pointers)

```
Time  : O(n log n)
Space : O(1)
```

This is the expected interview and LeetCode optimal solution.

Sources

Trapping Rain Water

Trapping Rain Water (LeetCode 42)

1. Problem Explanation (Simple English)

What is given?

An array `height`.

```
height[i]
```

represents the height of a building/bar.

Example:

```
height = [0,1,0,2,1,0,1,3,2,1,2,1]
```

What do we need to find?

After rain, water gets trapped between taller bars.

Return the total amount of trapped water.

Example:

```
Input:
[0,1,0,2]

Water trapped:
1 unit
```

Key Observation

Water above index `i` depends on:

```
Highest bar on left
Highest bar on right
```

Water at index `i`:

```
min(leftMax,rightMax) - height[i]
```

if positive.

---

2. Brute Force Approach

Idea

For every index:

Step 1

Find tallest bar on left.

Step 2

Find tallest bar on right.

Step 3

Calculate:

```C++
water =
min(leftMax,rightMax)
-
height[i]
```

Add to answer.

Time Complexity

For every index:

```
Find left max  -> O(n)
Find right max -> O(n)
```

Total:

```
O(n²)
```

Space Complexity

```
O(1)
```

C++ Code

C++

```cpp
class Solution {
public:
    int trap(vector<int>& height) {

        int n = height.size();

        int ans = 0;

        for(int i = 0; i < n; i++) {

            int leftMax = 0;
            int rightMax = 0;

            for(int j = 0; j <= i; j++) {
                leftMax = max(leftMax, height[j]);
            }

            for(int j = i; j < n; j++) {
                rightMax = max(rightMax, height[j]);
            }

            ans += min(leftMax, rightMax) - height[i];
        }

        return ans;
    }
};
```

---

3. Better Approach

Idea (Prefix Max + Suffix Max)

Observation

Instead of recomputing:

```
leftMax
rightMax
```

for every index,

precompute them.

Build Left Max

```
leftMax[i]
=
maximum height from 0 to i
```

Build Right Max

```
rightMax[i]
=
maximum height from i to n-1
```

Formula

For every index:

```
</> C++

water =
min(leftMax[i], rightMax[i])
-
height[i]
```

Time Complexity

Building arrays:

```
O(n)
```

Traversal:

```
O(n)
```

Total:

```
O(n)
```

Space Complexity

```
O(n)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    int trap(vector<int>& height) {

        int n = height.size();

        vector<int> leftMax(n);
        vector<int> rightMax(n);

        leftMax[0] = height[0];

        for(int i = 1; i < n; i++) {
            leftMax[i] = max(leftMax[i - 1], height[i]);
        }

        rightMax[n - 1] = height[n - 1];

        for(int i = n - 2; i >= 0; i--) {
            rightMax[i] = max(rightMax[i + 1], height[i]);
        }

        int ans = 0;

        for(int i = 0; i < n; i++) {
            ans += min(leftMax[i], rightMax[i]) - height[i];
        }

        return ans;
    }
};
```

---

4. Optimal Approach

Core Observation

We don't actually need two arrays.

Use two pointers:

```cpp
left = 0
right = n-1
```

Maintain:

```
leftMax
rightMax
```

Key Trick

If

```C++
leftMax < rightMax
```

Then water at `left` depends only on:

```
leftMax
```

because a taller wall already exists on the right.

So process left side.

Else

Process right side.

Why Does This Work?

Water formula:

```
min(leftMax,rightMax)
```

If:

```
leftMax < rightMax
```

then:

```
min(leftMax,rightMax)
=
leftMax
```

Therefore water on left side is already determined.

Algorithm

If

```cpp
height[left] < height[right]
```

Update:

```cpp
leftMax
```

If:

```cpp
height[left] < leftMax
```

water trapped:

```cpp
leftMax - height[left]
```

Move:

```cpp
left++
```

Else

Do the same for right side.

Time Complexity

Single traversal:

```
O(n)
```

Space Complexity

```
O(1)
```

C++ Code

```C++
```

```cpp
class Solution {
public:
    int trap(vector<int>& height) {

        int n = height.size();

        int left = 0;
        int right = n - 1;

        int leftMax = 0;
        int rightMax = 0;

        int ans = 0;

        while(left < right) {

            if(height[left] < height[right]) {

                if(height[left] >= leftMax) {
                    leftMax = height[left];
                }
                else {
                    ans += leftMax - height[left];
                }

                left++;
            }
            else {

                if(height[right] >= rightMax) {
                    rightMax = height[right];
                }
                else {
                    ans += rightMax - height[right];
                }

                right--;
            }
        }

        return ans;
    }
};
```

---

5. Dry Run

Input:

```
height =
[0,1,0,2,1,0,1,3,2,1,2,1]
```

Initial

```
left = 0
right = 11

leftMax = 0
rightMax = 0
```

Move Left

```
height[0] = 0

leftMax = 0
```

Water:

```
0
```

Next

```
height[1] = 1

leftMax = 1
```

Next

```
height[2] = 0

leftMax = 1
```

Water trapped:

```
1 - 0 = 1
```

Answer:

```
1
```

Next

```
height[3] = 2

leftMax = 2
```

Continue similarly...

Final answer:

```
6
```

What Beginners Usually Miss

Observation 1

Water at index `i` is:

```C++
min(leftMax,rightMax)
-
height[i]
```

NOT:

```C++
max(leftMax,rightMax)
-
height[i]
```

Observation 2

The prefix/suffix solution is already `O(n)`.

Most candidates stop there.

Observation 3 (Most Important)

When:

```
leftMax < rightMax
```

the left side is completely determined.

You do NOT need to know the exact future right maximum.

This observation allows removing the extra arrays.

Final Complexity

Brute Force

```
Time  : O(n²)
Space : O(1)
```

Better (Prefix + Suffix)

```
Time  : O(n)
Space : O(n)
```

Optimal (Two Pointers)

```
Time  : O(n)
Space : O(1)
```

This two-pointer solution is the expected interview and LeetCode optimal solution.

Sources